

Joint API

This is the API reference to the open source JointJS core library. If you're looking for the Rappid diagramming toolkit documentation, you can find that [here](#).

JointJS library exports three global variables: `joint`, `v` and `g`.

The `joint` namespace contains all the objects that you will use to build your diagrams. Additionally, `joint.version` property tells you which version of JointJS you're using.

The `v` global is lightweight SVG library that we call "Vectorizer". This tiny library makes manipulation with SVG documents much easier. JointJS uses this library internally. Normally, you don't have to get in touch with this library at all but for advanced uses, it can be handy.

The `g` global is another lightweight library used internally by JointJS that provides many useful geometry operations. Again, you might not get in touch with this library but when you do have the need to perform geometric operations in your applications, you'll certainly find it helpful.

dia.attributes

The attributes in JointJS define how the graphics elements are to be rendered inside of the element and link views. All the standard SVG [styling properties](#) are available (both `kebab-case` and `camelCase` styles). In addition JointJS defines new so-called "special" attributes and allows programmers to define their own. Here is the list of all built-in attributes.

dia.attributes.event

The `event` attribute makes the selected node and its descendants trigger an arbitrary event when clicked (mousedown/touchstart). This event is triggered on the view itself and the paper. The paper handler is called with the signature `cellView, evt, x, y`, while the cell view handler is called only with `evt, x, y`.

```
element.attr({
  image: {
    // pointerdown on the image SVG node will trigger the `element:delete` event
    event: 'element:delete',
    xlinkHref: 'trash.png'
    width: 20,
    height: 20
  }
});
// Binding handler to the event
paper.on('element:delete', function(elementView, evt) {
  // Stop any further actions with the element view e.g. dragging
  evt.stopPropagation();
  if (confirm('Are you sure you want to delete this element?')) {
    elementView.model.remove();
  }
});
```

dia.attributes.fill

The `fill` attribute becomes a special attribute only in case it's defined as an object, instead of the usual SVG syntax (e.g. `"#ffaabb"`). If it's defined as an object, it is assumed to be a gradient definition and must have the form defined by the [defineGradient\(\)](#) paper method.

```

element.attr('rect/fill', {
  type: 'linearGradient',
  stops: [
    { offset: '0%', color: '#E67E22' },
    { offset: '20%', color: '#D35400' },
    { offset: '40%', color: '#E74C3C' },
    { offset: '60%', color: '#C0392B' },
    { offset: '80%', color: '#F39C12' }
  ]
});

```

dia.attributes.filter

The `filter` attribute becomes a special attribute only in case it's defined as an object, instead of the usual SVG syntax (e.g. `"url(#myfilter)"`). If it's defined as an object, it must have the form defined by the `filterGradient()` paper method.

```

element.attr('rect/filter', {
  name: 'dropShadow',
  args: {
    dx: 2,
    dy: 2,
    blur: 3
  }
});

```

dia.attributes.magnet

When set to `true`, the subelement can become a source/target of a link during link reconnection. Useful for so called 'ports'.

dia.attributes.port

An object containing at least an `id` property. This property uniquely identifies the port. If a link gets connected to a `magnet` that has also a `port` object defined, the `id` property of the port object will be copied to the `port` property of the source/target of the link.

dia.attributes.ref

CSS selector pointing to an element that is used as a reference for relative positioning attributes.

dia.attributes.refCx

Set `cx` attribute of the subelement relatively to the width of the element referenced to by the selector in `ref` attribute. If the value is in the `[0, 1]` interval (or expressed in percentages, e.g. `'80%'`), the `cx` of the subelement will be set as a percentage of the width of the referenced element. If the value is `<0` or `>1`, the height of the subelement will be smaller/bigger than the width of the referenced element by the amount specified. Note that this makes sense only for SVG elements that support `rx` and `ry` attributes, such as `<ellipse>`.

dia.attributes.refCy

Set `cy` attribute of the subelement relatively to the height of the element referenced to by the selector in `ref` attribute. If the value is in the `[0, 1]` interval (or expressed in percentages, e.g. `'80%'`), the `cy` of the subelement will be set as a percentage of the height of the referenced element. If the value is `<0` or `>1`, the height of the subelement will be smaller/bigger than the height of the referenced element by the amount specified. Note that this makes sense only for SVG elements that support `rx` and `ry` attributes, such as `<ellipse>`.

dia.attributes.refDx

alias: **ref-dx**

Make x-coordinate of the subelement relative to the right edge of the element referenced to by the selector in `ref` attribute.

dia.attributes.refDy

Make y-coordinate of the subelement relative to the bottom edge of the element referenced to by the selector in `ref` attribute.

dia.attributes.refHeight

alias: **ref-height**

Set height of the subelement relatively to the height of the element referenced to by the selector in `ref` attribute. If the value is in the `[0, 1]` interval (or expressed in percentages, e.g. `'80%'`), the height of the subelement will be set as a percentage of the height of the referenced element. If the value is `<0` or `>1`, the height of the subelement will be smaller/bigger than the height of the referenced element by the amount specified. Note that this makes sense only for SVG elements that support `width` and `height` attributes, such as `<rect>`.

dia.attributes.refRx

Set `rx` attribute of the subelement relatively to the width of the element referenced to by the selector in `ref` attribute. If the value is in the `[0, 1]` interval (or expressed in percentages, e.g. `'80%'`), the `rx` of the subelement will be set as a percentage of the width of the referenced element. If the value is `<0` or `>1`, the height of the subelement will be smaller/bigger than the width of the referenced element by the amount specified. Note that this makes sense only for SVG elements that support `rx` and `ry` attributes, such as `<ellipse>`.

dia.attributes.refRy

Set `ry` attribute of the subelement relatively to the height of the element referenced to by the selector in `ref` attribute. If the value is in the `[0, 1]` interval (or expressed in percentages, e.g. `'80%'`), the `ry` of the subelement will be set as a percentage of the height of the referenced element. If the value is `<0` or `>1`, the height of the subelement will be smaller/bigger than the

height of the referenced element by the amount specified. Note that this makes sense only for SVG elements that support `rx` and `ry` attributes, such as `<ellipse>`.

dia.attributes.refWidth

alias: **ref-width**

Set width of the subelement relatively to the width of the element referenced to by the selector in `ref` attribute. If the value is in the `[0, 1]` interval (or expressed in percentages, e.g. `'80%'`), the width of the subelement will be set as a percentage of the width of the referenced element. If the value is `<0` or `>1`, the width of the subelement will be smaller/bigger than the width of the referenced element by the amount specified. For example, `'ref-width': .75` sets the width of the subelement to `75%` of the width of the referenced element. `'ref-width': 20` makes the width to be `20px` less than the width of the referenced element. Note that this makes sense only for SVG elements that support `width` and `height` attributes, such as `<rect>`.

dia.attributes.refX

alias: **ref-x**

Make x-coordinate of the subelement relative to the x-coordinate of the element referenced to by the selector in `ref` attribute. If `ref-x` is in the `(0,1)` interval (or expressed in percentages, e.g. `'80%'`), the offset is calculated from the fraction of the bounding box of the referenced element. Otherwise, it is an absolute value in pixels.

dia.attributes.refX2

Same as `refX`. Useful when one needs to position an element absolutely and relatively at the same time.

```
{ refX: '50%', refX2: 20 } // moves the element by 50% of the referenced width plu
```

dia.attributes.refY

alias: **ref-y**

Make y-coordinate of the subelement relative to the y-coordinate of the element referenced to by the selector in `ref` attribute. If `ref-y` is in the `(0,1)` interval (or expressed in percentages, e.g. `'80%'`), the offset is calculated from the fraction of the bounding box of the referenced element. Otherwise, it is an absolute value in pixels.

dia.attributes.refY2

Same as `refY`. Useful when one needs to position an element absolutely and relatively at the same time.

```
{ refY: '50%', refY2: 20 } // moves the element by 50% of the referenced height pl
```

dia.attributes.resetOffset

If set to `true`, the subelement offset (the distance from x0 y0 to the most top-left point) will be reset to x0 y0.

```
element.attr({
  path: {
    // The path bbox for `d="M 10 10 20 20"` is x10 y10 w10 h10.
    d: 'M 10 10 20 20',
    // The path bbox will be changed to x0 y0 w10 h10.
    // This has same effect as passing path with `d="M 0 0 10 10"`
    resetOffset: true
  }
});
```

dia.attributes.sourceMarker

Valid for `<path>` subelements. It draws an SVG element at the beginning of a path. The element is automatically rotated based on the path direction. It's defined as an object with `type` property and any other visual attributes. The valid values for `type` are `'path'`, `'circle'`, `'ellipse'`, `'rect'`, `'polyline'` and `'polygon'`.

```
link.attr('.connection/sourceMarker', {
  type: 'circle', // SVG Circle
  fill: '#666',
  stroke: '#333',
  r: 5, // radius of the circle
  cx: 5 // move the centre of the circle 5 pixels from the end of the path
});
```

dia.attributes.stroke

The `stroke` attribute becomes a special attribute only in case it's defined as an object. This has the exact same behaviour as the `fill` attribute.

dia.attributes.style

An object containing CSS styles for a subelement.

dia.attributes.targetMarker

Valid for `<path>` subelements. The same as `sourceMarker` but draws an SVG element at the end of the path. Note the coordinate system for drawing is rotated by 180 degrees for convenience (this allows to use the same marker for source and target and have them both face the connected elements).

dia.attributes.text

Valid only for `<text>` subelements. `text` attribute contains the text that will be set either directly to the `<text>` subelement or its `<tspan>` children depending on whether the text is multiline or not

(contains `'\n'` character(s)).

dia.attributes.textPath

Valid only for `<text>` subelements. `textPath` can be either a string or an object. If it is a string, it specifies a path the text should go along. If it is an object, then it should contain a `d` property that specifies the path the text should go along and optionally other attributes that will be set on the automatically created `<textPath>` SVG element such as `startOffset`. See [text\(\)](#) method of Vectorizer objects for more details.

dia.attributes.textWrap

Valid for `<text>` subelements. Similar to `text` attribute but in this case the text is automatically wrapped to fit within the reference bounding box. It's defined as an object with `text` property and optional `width` and `height`, which can adjust the final size of the wrapped text. Positive values increment and negative values decrement the dimension. Define values as percentage when a fraction of the dimension is required.

```
textWrap: {
  text: 'Here is the text to be wrapped.',
  width: -10, // reference width minus 10
  height: '50%' // half of the reference height
}
```

dia.attributes.vertexMarker

Valid for `<path>` subelements. The same as [sourceMarker](#) but draws SVG elements at every vertex of the path.

dia.attributes.xAlignment

alias: **x-alignment**

If set to `'middle'`, the subelement will be centered around its new x-coordinate.

dia.attributes.yAlignment

alias: **y-alignment**

If set to `'middle'`, the subelement will be centered around its new y-coordinate.

dia.Cell

The basic model for diagram cells. It's a [Backbone model](#) with a few additional properties and methods. The first one to mention is a unique identifier for the cell. Each cell has a [unique ID](#) that is stored in the `id` property.

dia.Cell.define

`define`(type [, defaultAttributes, prototypeProperties, staticPropert

Helper to define new Cell class or extend the existing one. The `type` must be a unique identifier of the class, which determines the location of the class definition in the `joint.shapes` namespace (the type is the path to the class definition delimited by dots (`.`)). When creating an instance of the cell, any unspecified attributes will be set to the value from the `defaultAttributes`.

```
// Define a new Ellipse class in `joint.shapes.examples` namespace.
var Ellipse = joint.dia.Element.define('examples.Ellipse', {
  markup: '<ellipse/>',
  attrs: {
    ellipse: {
      fill: 'white',
      stroke: 'black',
      refRx: .5,
      refRy: .5,
      refCx: .5,
      refCy: .5
    }
  }
});
// `Ellipse === joint.shapes.examples.Ellipse` now
// Instantiate an element
var el1 = (new Ellipse).position(100,100).size(120,50).addTo(graph);
```

```
// Define a new ColoredEllipse class that inherits from Ellipse.
var BlueEllipse = Ellipse.define('examples.ColoredEllipse', {
  attrs: {
    ellipse: {
      fill: 'black'
    }
  }
}, {
  // Prototype methods
  changeColor: function() {
    return this.attr('ellipse/stroke', this.constructor.getRandomColor());
  }
}, {
  // Static methods
  getRandomColor: function() {
    return '#' + Math.floor(Math.random() * 16777215).toString(16);
  }
});

var el2 = (new ColoredEllipse).changeColor().position(300,100).size(120,50).addTo(
```

dia.CellView

The view for the `joint.dia.Cell` model. It inherits from `Backbone.View` and is responsible for:

- Rendering a cell inside of a paper
- Handling the cell's pointer events
- Provides various methods for working with the cell (visually)

To find the view associated with a specific cell (model), use the `findViewByModel` method of the paper. For example:

```
var cellView = paper.findViewByModel(cell);
```

dia.CellView.prototype.highlight

```
cellView.highlight([el[, options]])
```

Highlights the cell view.

Arguments:

- **el** - if not provided, then the cell view's `$el` will be used
- **options**:
 - **highlighter**:
 - **name** - the name of the highlighter (see [highlighters](#))
 - **options** - an options object that will be passed directly to the highlighter specified by `name`
 - **type** - the kind of highlighting being done (embedding, connecting, magnetAvailability, or elementAvailability)

dia.CellView.prototype.unhighlight

```
cellView.unhighlight([el[, options]])
```

Removes the highlighting being done to the cell.

It is **important** to note that if you highlighted a cell using custom options, you must provide those exact same options when using the unhighlight method.

dia.Element

The model for diagram elements. It inherits from [joint.dia.Cell](#) with a few additional properties and methods specific to elements. These properties can be put into three groups:

Geometry

Coordinates of an element are stored in the `position` property that is an object with `x` and `y` keys. `position` can be accessed or set directly using the regular Backbone `set()`/`get()` methods or through the [translate](#) method.

Rotation angle is stored in the `angle` property. This angle is in degrees and the rotation origin is always considered to be the center of the element. `angle` can be also accessed or set directly using the regular Backbone `set()`/`get()` methods or through the [rotate](#) method.

Size of an element is stored in the `size` property that is an object with `width` and `height` keys. Again, `size` can be accessed or set directly using the regular Backbone `set()`/`get()` methods or through the [resize](#) method.

Presentation

Another important property is `attrs` which is an object with keys representing **selectors** that match subelements and values which are SVG attributes that will be set on the subelements. One can find a list of **SVG attributes** and their descriptions e.g. on [MDN](#).

It is important to note that each `joint.dia.Element` defines an SVG markup which is then used by `joint.dia.ElementView` to render the element to the paper. For instance, the `joint.shapes.basic.Rect` element (that inherits from `joint.dia.Element`) defines its markup as follows:

```
<g class="rotatable"><g class="scalable"><rect/></g><text/></g>
```

Therefore, in order to set a red fill color for the rectangle subelement, the `attrs` object should contain:

```
rect: { fill: 'red' }
```

Again, it is not recommended to change the `attrs` object directly. Instead, use the `attr` method.

The `z` property specifies the stack order of the element in the SVG DOM. An element with a higher `z` level is in front of an element with a lower `z` level. (This also stands for links which have the exact same property.)

Nesting

The last two properties of elements are `embeds` and `parent`. These two are related to elements that contain or are contained within other elements forming a hierarchical structure. `embeds` is a list of cell `id`'s that are embedded inside the element. `parent` is an `id` of the parent element of the embedded one. When a parent element is translated, all its children get translated too.

dia.Element.events

The following list contains events that you can react on:

- `change` - generic event triggered for any change on the element - *fn(element, opt)*
- `change:position` - triggered when the element changes its position - *fn(element, newPosition, opt)*
- `change:angle` - triggered when the element gets rotated - *fn(element, newAngle, opt)*
- `change:size` - triggered when the element gets resized - *fn(element, newSize, opt)*
- `change:attrs` - triggered when the element changes its attributes - *fn(element, newAttrs, opt)*
- `change:embeds` - triggered when other cells were embedded into the element - *fn(element, newEmbeds, opt)*
- `change:parent` - triggered when the element got embedded into another element - *fn(element, newParent, opt)*
- `change:z` - triggered when the element is moved in the z-level (`toFront` and `toBack`) - *fn(element, newZ, opt)*
- `transition:start` - triggered when a transition starts. - *fn(element, pathToAttribute)*
- `transition:end` - triggered when a transition ends. - *fn(element, pathToAttribute)*

```
// Listening for changes of the position to a single element
element1.on('change:position', function(element1, position) {
  alert('element1 moved to ' + position.x + ',' + position.y);
});
// All elements events are also propagated to the graph.
graph.on('change:position', function(element, position) {
  console.log('Element ' + element.id + 'moved to ' + position.x + ',' + position.y);
});
// Using the option parameter and a custom attribute
graph.on('change:custom', function(element, custom, opt) {
  if (opt.consoleOutput) {
    console.log('Custom attribute value changed to ' + custom + '');
  }
});
element2.prop('custom', 'myValue', { consoleOutput: true });
```

dia.Element.ports

Ports

Many diagramming applications deal with elements with ports. Ports are usually displayed as circles inside diagram elements and are used not only as "sticky" points for connected links but they also

further structure the linking information. It is common that certain elements have lists of input and output ports. A link might then point not to the element as a whole but to a certain port instead.

It's easy to add ports to arbitrary shapes in JointJS. This can be done either by passing a ports definition as an `option` in the constructor or using the ports API to get/add/remove single or multiple ports. For more information on how to define ports please see [Port configuration](#) section.

Port API on `joint.dia.Element`

- `hasPort` / `hasPorts`
- `addPort` / `addPorts`
- `removePort`
- `getPort` / `getPorts`
- `portProp`
- `getPortPositions`

Port configuration

```
// Single port definition
var port = {
  id: 'abc',
  group: 'a',
  args: {},
  label: {
    position: {
      name: 'top',
      args: {}
    },
    markup: '<text class="label-text" fill="blue"/>'
  },
  attrs: { text: { text: 'port1' } },
  markup: '<rect width="10" height="10" stroke="red"/>'
};

// a.) add a port in constructor.
var rect = new joint.shapes.basic.Rect({
  position: { x: 50, y: 50 },
  size: { width: 90, height: 90 },
  ports: {
    groups: {},
    items: [ port ]
  }
});

// b.) or add a single port using API
rect.addPort(port);
```

id	string	It is automatically generated if no <code>id</code> provided. If with the same port id are therefore not allowed
group	string	Group name, more info in groups section.
args	object	Arguments for the port layout function. Available could be found in layout.Port .
attrs	object	JointJS style attribute definition. The same notati
markup	string	Custom port markup. Multiple roots are not allo <pre><g> <rect class="outer" width="15" heiç <rect class="inner" width="15" heiç </g></pre> It defaults to <code><circle class="joint-port-body"</code>

label	object	Port label layout configuration. E.g. label position be found in <code>layout.PortLabel</code> section.
label.position	string object	Port label position configuration. It could be a <code>string</code> settings or an <code>object</code> where it's possible to set <pre>{ position: 'left'} // or ... { position: { name: 'left', args: { dx: 10 } } }</pre>
- label.position.name	string	Stands for the layout type, match the layout method <code>name: 'left'</code> is implemented as <code>joint.layout.P</code>
- label.position.args	object	Additional arguments for the layout method. Available could be found in <code>layout.PortLabel</code> section.
label.markup	string	Custom port label markup. Multiple roots are not allowed. <code>fill="#000000"/></code> .
z	number string	Alternative to HTML <code>z-index</code>. <code>z</code> sets the position Shapes most likely consist of 1 or more DOM elements placed into the element <code>rotatable</code> group (if the group element <code>elementView.el</code> is used for the parent last element in the <code>rotatable</code> group. Ports with the position (index within the children of the container <code>z: 'auto'</code> are taken into account) equals to <code>z</code>. For instance an element with the following markup <pre><g class="rotatable"> <g class="scalable"><rect/></g> <text/> </g></pre> will be rendered like this: <pre><g model-id="element1"> <g class="rotatable"> <g class="port"></g> <g class="scalable"><rect/></g> <g class="port"></g></pre>

```
<text/>
  <g class="port"></g>
</g>
</g>
```

Another example with simplified markup `<circle`

```
<g model-id="element2">
  <g class="port"></g>
  <circle/>
  <g class="port"></g>
  <g class="port"></g>
  <text/>
  <g class="port"></g>
  <g class="port"></g>
  <g class="port"></g>
  <g class="port"></g>
  <g class="port"></g>
  <g class="port"></g>
</g>
```

All properties described above are optional and everything has own default. E.g. `element.addPorts([{}], {})` will add 2 ports with default settings.

Port groups configuration

`group` attribute comes to play when you're not happy with the default port alignment. It's also handy when you need to define multiple ports with similar properties. `group` defines defaults for ports belonging to the group. Any `group` property can be overwritten by a port in this group except the type of layout - `position`. 'group' defines the layout and port 'args' are the only way how a port can affect it.

```
// Define ports and port groups in element constructor.
var groupA;
var rect = new joint.shapes.basic.Rect({
  // ...
  ports: {
    groups: {
      'group1': groupA,
      // 'group2': ...,
      // 'group3': ...,
    },
    items: []
  }
});

groupA = {
  position: {
    name: 'string', // layout name
    args: {}, // arguments for port layout function, properties depend
  },
  label: {
    // ....
  },
  attrs: {},
  markup: '<rect width="10" height="10" stroke="red"/>'
};
```

position	string object	Port position configuration. Could be <code>string</code> to set default settings or <code>object</code> where is possible to set
• position.name	string	Stands for the layout type, match the layout method <code>joint.layout.Port</code> namespace: <code>name: 'left'</code> is in <code>joint.layout.Port.left</code> .
	object	Arguments for the port layout function. Available p

• position.args		of layout. More information could be found in lay
attrs	<i>object</i>	JointJS style attribute definition. The same notation Element .
markup	<i>string</i>	Custom port markup. Multiple roots are not allowe <pre><g> <rect class="outer" width="15" height="15" fill="blue"/> <rect class="inner" width="15" height="15" fill="red"/> </g></pre> . It defaults to <code><circle class="joint-port-body" r="15" stroke="#000000"/></code>
label	<i>object</i>	Port label layout configuration. E.g. label position, about port label layouts could be found in layout
• label.position	<i>string object</i>	Port label position configuration. It could be a <code>string</code> type directly with default settings or an <code>object</code> with type and options. <pre>{ position: 'left' } // or ... { position: { name: 'left', args: { dx: 10 } } }</pre>
◦ label.position.name	<i>string</i>	Stands for the layout type, match the layout method <code>joint.layout.PortLabel</code> namespace: <code>name: 'left'</code> <code>joint.layout.PortLabel.left</code> .
◦ label.position.args	<i>object</i>	Additional arguments for the layout method. Available layout type. More information could be found in l
• label.markup	<i>string</i>	Custom port label markup. Multiple roots are not allowed. <code>class="joint-port-label" fill="#000000"/></code> .

Custom markup

Both port and port label can have custom markup.

```
var rect = new joint.shapes.basic.Rect({
  // ...
});

rect.addPort({ markup: '<rect width="10" height="10" fill="blue"/>' });
rect.addPort({ markup: '<rect width="15" height="15" fill="red"/>', label: { m
```

or, it can be set as an default port markup/port label markup on an element model:

```
var rect = new joint.shapes.basic.Rect({
  portMarkup: '<rect width="20" height="20" fill="black"/>',
```

```
portLabelMarkup: '<text fill="yellow"/>',
// ...
});
```

dia.Element.prototype.addPort

```
element.addPort(port, [opt])
```

Add a single port, where `port` could be defined as described in section [Port interface](#)

dia.Element.prototype.addPorts

```
element.addPorts(ports, [opt])
```

Add array of `ports`. Ports are validated for `id` duplicities, if there is a id collision, no new ports are added, where `port` could be defined as describe in section [Port interface](#)

dia.Element.prototype.addTo

```
element.addTo(graph)
```

Add the element to the `graph` (an instance of `joint.dia.Graph`). This is equivalent to calling `graph.addCell(element)`.

dia.Element.prototype.attr

```
element.attr(attrs)
```

Set SVG attributes (and JointJS special attributes) on subelements. `attr` can either be an object or string representing a path to a nested attribute. If it is an object, the keys of the `attrs` object are CSS selectors matching the subelements. The values are objects containing SVG attributes and their values. `attrs` object will be mixed with `attrs` property of the `element` model. This is a convenient way of rewriting only some of the attributes of the subelements. For overwriting all attributes of all subelements, use `element.set(attrs)`.

```
element.attr({
  rect: { fill: 'blue' },
  text: { fill: 'white', 'font-size': 15 },
  '.myrect2': { fill: 'red' }
});
```

An alternative call using a string path and a value:

```
element.attr('text/font-size', 12);
```

dia.Element.prototype.clone

```
element.clone(options)
```

Returns a new instance of the element with identical attributes. If `options.deep === true`, then all the embedded cells (elements, links) of the element are cloned as well. In this case, the return value is an array of instances rather than a single instance.

dia.Element.prototype.embed

```
element.embed(cell)
```

Embed a cell (element or link) into the element. The element then becomes a parent of the embedded cell. When a parent is moved (translated), all cells embedded into that parent will move as well. If links are embedded, their vertices move with the parent. This way both options are available: if a link is not embedded but its source/target elements are and their parent moves, the embedded elements move with the parent but the link vertices stay at the same position. If the link is embedded with its source/target elements, its vertices move as the parent moves.

dia.Element.prototype.findView

```
element.findView(paper)
```

Find view (`joint.dia.ElementView`) for the element model in the `paper`. This is a shortcut to the equivalent call `paper.findViewByModel(element)`

dia.Element.prototype.fitEmbeds

```
element.fitEmbeds([opt])
```

Resize the element so that it fits all the embedded elements inside it. If `opt.deep` is `true`, the resizing will be done recursively for any embedded elements that contain embedded elements themselves. Set `opt.padding` if you want certain padding on all the parent elements.

dia.Element.prototype.getAncestors

```
element.getAncestors()
```

Return an array of all the ancestors of this element starting from the immediate parent all the way up to the most distant ancestor.

dia.Element.prototype.getBBox

```
element.getBBox()
```

Returns an element's bounding box represented as a `g.rect` object (see [geometry library](#)).

```
if (element1.getBBox().intersect(element2.getBBox())) {
  // elements intersect
}
```

dia.Element.prototype.getEmbeddedCells

```
element.getEmbeddedCells([opt])
```

Return an array of all the embedded cells of an element. If all you need is `id`'s of all the embedded cells, use `element.get('embeds')`. If `opt.deep` is `true`, all the deeply embedded cells will be returned. The order in which the cells are returned depends on the search algorithm used. By default, [Depth-first search \(DFS\)](#) algorithm is used. If `opt.breadthFirst` is `true`, the [Breadth-first search](#) algorithm will be used instead.

dia.Element.prototype.getPort

```
element.getPort(id)
```

Returns the port specified by an `id`. If such a port does not exist the method returns `undefined`.

dia.Element.prototype.getPortIndex

```
element.getPortIndex(portId)
```

Returns the port index in the array of port items.

dia.Element.prototype.getPorts

```
element.getPorts()
```

Returns an array of all ports on the element. If there is no port an empty array is returned.

dia.Element.prototype.getPortsPositions

```
element.getPortsPositions(groupName)
```

Returns the positions and the angle of all ports in the group, relatively to the element position.

dia.Element.prototype.getTransitions

```
element.getTransitions()
```

Return an array of all active transitions (their paths).

dia.Element.prototype.hasPort

```
element.hasPort(id)
```

Check if an element contains a port with `id`. Returns `boolean`.

dia.Element.prototype.hasPorts

```
element.hasPorts()
```

Check if an element has any ports defined. Returns `boolean`.

dia.Element.prototype.isElement

```
element.isElement()
```

Always returns `true`. The reason the method is here is that both `joint.dia.Element` and `joint.dia.Link` inherit from `joint.dia.Cell`. This method is useful if you don't know what the cell is. Calling `el.isElement()` is equivalent to `el instanceof joint.dia.Element`. Example:

```
var cell = graph.getCell(myId)
if (cell.isElement()) {
  // Do something if the cell is an element.
}
```

dia.Element.prototype.isEmbeddedIn

`element.isEmbeddedIn(element [, opt])`

Return `true` if the element is embedded in another element `element`. If `opt.deep` is `false`, only direct parentage will be checked. `opt.deep` is `true` by default.

dia.Element.prototype.isLink

`element.isLink()`

Always returns `false`. The reason the method is here is that both `joint.dia.Element` and `joint.dia.Link` inherit from `joint.dia.Cell`. This method is useful if you don't know what the cell is. Calling `e1.isLink()` is equivalent to `e1 instanceof joint.dia.Link`. Example:

```
var cell = graph.getCell(myId)
if (cell.isLink()) {
  // Do something if the cell is a link.
}
```

dia.Element.prototype.portProp

`element.portProp(portId, path, [value])`

Set properties, possibly nested, on the element port. This is an equivalent of the `attr()` method but this time for custom data properties.

```
element.portProp('port-id', 'attrs/circle/fill', 'red');
element.portProp('port-id', 'attrs/circle/fill'); // 'red'

element.portProp('port-id', 'attrs/circle', { r: 10, stroke: 'green' });
element.portProp('port-id', 'attrs/circle'); // { r: 10, stroke: 'green', fill: 'r
```

dia.Element.prototype.position

`element.position(x, y, [opt])`

Set the element position to `x` and `y` coordinates. This is almost equivalent to `element.set('position', { x: x, y: y }, opt)`. However, this method provides some additional functionality. If you set `opt.parentRelative` flag to `true`, the `x` and `y` coordinates will be treated relatively to the parent element of this element. If `position()` is called without arguments, it returns the current position. If `position({ parentRelative: true })` is called without `x` and `y` coordinates and with the `parentRelative` flag set to `true`, the method returns the current position of the element relative to its parent. Setting `opt.deep` to `true` will position not only the element but also all its descendants keeping the original distances from a child to the element origin.

```
e11.position(100, 100);
e11.embed(e12);
e12.position(10, 10, { parentRelative: true });
e12.position() // --> 110@110
```

```
el1.position(200,200, { deep: true });
el2.position() // --> 210@210
```

dia.Element.prototype.prop

`element.prop(properties)`

Set properties, possibly nested, on the element model. This is an equivalent of the `attr()` method but this time for custom data properties.

```
element.prop('name/first', 'John')
element.prop('name/first') // 'John'
element.prop({ name: { first: 'John' } })
// Nested arrays are supported too:
element.prop('mylist/0/data/0/value', 50)
element.prop({ mylist: [ { data: [ { value: 50 } ] } ] })
```

dia.Element.prototype.remove

`element.remove(options)`

Remove the element from the graph. All its embedded elements will get removed too and the element gets unembedded from its parent element. By default, all the associated links are removed too. To suppress this behaviour, set `options.disconnectLinks === true`. In this case, all the associated links get disconnected from this element rather than removed completely from the graph.

dia.Element.prototype.removeAttr

`element.removeAttribute(path, [options])`

Remove a previously set attribute from the element. `path` can either be a string that specifies the path to the, possibly nested, attribute to be removed or an array of more paths. The associated element view makes sure the element gets re-rendered properly. If `options` is passed, it can contain data that is passed over to the event listeners for the `change:attrs` event triggered on the element itself and also on the graph the element is in.

dia.Element.prototype.removePort

`element.removePort(port, [opt])`

Remove a port from an element, where `port` is a port object, or `portId`.

dia.Element.prototype.resize

`element.resize(width, height [, opt])`

Resize an element in place so that the "scalable" group has width `width` and height `height`. In place in this case means that the top-left corner of the element stays at the same position after resizing. In other words, the element is stretched to the bottom/right (by default). To change the direction of resizing, set `opt.direction` to one of `'left'`, `'right'`, `'top'`, `'bottom'`, `'top-right'`, `'top-left'`, `'bottom-left'` or `'bottom-right'` (the default).

There is a difference between a classical scale and `resize` operations. `resize` doesn't actually scale the whole SVG `<g>` element grouping all its subelements. It only scales subelements of the `<g class="scalable">` group. This is very useful and brings a lot of flexibility in defining which subelements should be scaled and which not. Imagine a simple rectangle element with text inside. Usually, when we resize the whole element, we expect the rectangle to get scaled while the text should stay the same size, only its position should be adjusted so that the text stays in the center of the rectangle. This can be easily achieved by adding the `<rect/>` element to the `<g class="scalable">` group in the markup and positioning the text subelement relatively to the `<rect/>` element: `<text ref-x=".5" ref-y=".5" ref="rect" />`. Note that neither of `ref-x`, `ref-y` and `ref` attributes is an SVG standard attribute. These are special attributes introduced by JointJS. More on these in the section on [Special attributes](#).

dia.Element.prototype.rotate

```
element.rotate(deg, [absolute, origin, opt])
```

Rotate an element by `deg` degrees around its center. If the optional `absolute` parameter is `true`, the `deg` will be considered an absolute angle, not an addition to the previous angle. If `origin` is passed in the form of an object with `x` and `y` properties, then this point will be used as the origin for the rotation transformation.

dia.Element.prototype.scale

```
element.scale(sx, sy, origin[, opt])
```

Scales the element's position and size relative to the given origin.

dia.Element.prototype.stopTransitions

```
element.stopTransitions([path])
```

Stops all running transitions. If parameter `path` is provided, it will stop only transitions specified by this path.

dia.Element.prototype.toBack

```
element.toBack([opt])
```

Move the element so it is behind all other cells (elements/links). If `opt.deep` is `true`, all the embedded cells of this element will get higher `z` index than that of this element in a Breadth-first search (BFS) fashion. This is especially useful in hierarchical diagrams where if you want to send an element to the back, you don't want its children (embedded cells) to be hidden behind that element.

dia.Element.prototype.toFront

```
element.toFront([opt])
```

Move the element so it is on top of all other cells (element/links). If `opt.deep` is `true`, all the embedded cells of this element will get higher `z` index than that of this element in a Breadth-first search (BFS) fashion. This is especially useful in hierarchical diagrams where if you want to send an element to the front, you don't want its children (embedded cells) to be hidden behind that element.

All elements have a `z` property defining their z-level in the graph. This `z` property can even be set directly by `element.set('z', 123)`. This change will be automatically handled by the `joint.dia.Paper` object associated with the `joint.dia.Graph` object this element is part of and all the SVG elements will get resorted so that their position in the DOM reflects the `z` level.

dia.Element.prototype.toJSON

`element.toJSON()`

Return a copy of the element's attributes for JSON serialization. This can be used for persistence or serialization. Note that this method doesn't return a JSON string but rather an object that can be then serialized to JSON with `JSON.stringify()`.

dia.Element.prototype.transition

`element.transition(path, value [, opt])`

Allows to change the element's property gradually over a period of time. This method lets you specify what property to change (`path`), when the transition will start (`options.delay`), how long the transition will last (`options.duration`), how the transition will run (`options.timingFunction`), and how to interpolate the property value (`options.valueFunction`).

```
element.transition('position/x', 250, {
  delay: 100,
  duration: 500,
  timingFunction: function(t) { return t*t; },
  valueFunction: function(a, b) { return function(t) { return a + (b - a) * t; }
});
// will start changing the element's x-coordinate in 100ms, for period of 500ms.
```

JointJS comes pre-built with some common timing and interpolating functions. The timing functions are defined in the `joint.util.timing` namespace and the interpolating functions in the `joint.util.interpolate` namespace. The predefined timing functions are:

- `linear`
- `quad`
- `cubic`
- `inout`
- `exponential`
- `bounce`

and the predefined interpolating functions are:

- `number`
- `object`
- `hexColor`
- `unit`

```
element.transition('attrs/text/font-size', '1em', {
  valueFunction: joint.util.interpolate.unit,
  timingFunction: joint.util.timing.bounce
});
// will start changing the current font size value to 1em in the bounce fashion.
```

dia.Element.prototype.translate

`element.translate(tx, [ty], [opt])`

Translate an element by `tx` pixels in x axis and `ty` pixels in y axis. `ty` is optional in which case the translation in y axis will be considered zero. The optional options object `opt` can be used to pass additional parameters to the event handlers listening on `'change:position'` events.

`opt.transition` can be used to initiate an animated transition rather than a sudden move of the element. See [joint.dia.Element.transition](#) for more info on transitions. If `opt.restrictedArea` is set, the translation of the element will be restricted to that area only. The `restrictedArea` is an object of the form `{ x: Number, y: Number, width: Number, height: Number }`. This is useful, e.g. if you want to restrict the translation of an embedded element within its parent. The only thing you have to do in this case is to pass the bounding box of the parent element to the `restrictedArea` option:

```
myElement.translate(50, 50, { restrictedArea: graph.getCell(myElement.get('parent'))
```

The code above makes sure that the element `myElement` never crosses the bounding box of its parent element. note that this also works if the element `myElement` has other embedded elements. In other words, the bounding box of the `myElement` that is used to calculate the restriction is the total bounding box, including all its children (in case they are "sticking out").

[dia.Element.prototype.unembed](#)

```
element.unembed(cell)
```

Free up an embedded cell from its parent element.

[dia.ElementView](#)

The view for the [joint.dia.Element](#) model. It inherits from [joint.dia.CellView](#) and is responsible for:

- Rendering an element inside of a paper
- Handling the element's pointer events
- Provides various methods for working with the element (visually)

To find the view associated with a specific element (model), use the [findViewByModel](#) method of the paper.

```
var elementView = paper.findViewByModel(element);
```

[dia.ElementView.prototype.getBBox](#)

```
elementView.getBBox([opt])
```

Return a bounding box for an element view. If `opt.useModelGeometry` option is set to `true`, the resulting bounding box will be calculated based on the element model dimensions (so that SVG sub elements "sticking out" of the element will be excluded). The difference from [dia.Element.prototype.getBBox](#) is that in this case, the bounding box will be adjusted based on the `joint.dia.Paper` translate and scale.

[dia.Graph.constructor](#)

joint.dia.Graph is the model holding all the cells (elements and links) of the diagram. It's a [Backbone model](#). The collection of all the cells is stored in the property `cells`.

The graph is a powerful data model behind all JointJS diagrams. It not only provides an efficient storage for directed graphs but also offers useful algorithms for traversing the graphs.

Additionally, the graph accepts an option object in its constructor function that can contain the `cellNamespace` option. This option can be used to change the default behavior of JointJS which by default reads cell model definitions from the `joint.shapes` namespace. For example, if a cell is of type `'myshapes.MyElement'`, then the graph looks up `joint.shapes.myshapes.MyElement` model when deserializing a graph from the JSON format. If the graph is instantiated as e.g. `var graph = new joint.dia.Graph({}, { cellNamespace: myShapesNamespace })`, then the graph will read the model definition from the `myShapesNamespace.myshapes.MyElement` object instead. This is useful in situations where you don't want to - for any reason - use the `joint.shapes` namespace for defining your own custom shapes. This option is often used in combination with the `cellNamespaceView` option on the `joint.dia.Paper` object.

dia.Graph.events

The following list contains events that you can react on:

- `change` - generic event triggered for any change in the graph
- `add` - triggered when a new cell is added to the graph
- `remove` - triggered when a cell is removed from the graph
- Moreover, all the events that are triggered on elements or links are propagated to the graph as well.

```
graph.on('add', function(cell) {
  alert('New cell with id ' + cell.id + ' added to the graph.')
})
```

dia.Graph.JSON

The JointJS graph JSON representation has the following format:

```
{
  cells: [// Array of cells (ie. links and elements).
    {
      id: '3d90f661-fe5f-45dc-a938-bca137691eeb',// Some randomly generated
      type: 'basic.Rect',
      attrs: {
        'stroke': '#000'
      },
      position: {
        x: 0,
        y: 50
      },
      angle: 90,
      size: {
        width: 100,
        height: 50
      },
      z: 2,
      embeds: [
        '0c6bf4f1-d5db-4058-9e85-f2d6c74a7a30',
        'cdbcfe073-b160-4e8f-a9a0-22853f29cc06'
      ],
      parent: '31f348fe-f5c6-4438-964e-9fc9273c02cb'
      // ... and some other, maybe custom, data properties
    }
  ]
}
```

dia.Graph.prototype.addCell

```
graph.addCell(cell)
```

Add a new cell to the graph. If `cell` is an array, all the cells in the array will be added to the graph.

```
var rect = new joint.shapes.basic.Rect({
  position: { x: 100, y: 100 },
  size: { width: 70, height: 30 },
  attrs: { text: { text: 'my rectangle' } }
})
var rect2 = rect.clone()
var link = new joint.dia.Link({ source: { id: rect.id }, target: { id: rect2.id } })
var graph = new joint.dia.Graph
graph.addCell(rect).addCell(rect2).addCell(link)
```

dia.Graph.prototype.addCell

```
graph.addCell(cells[, opt])
```

```
graph.addCell(cell, cell, ..[, opt])
```

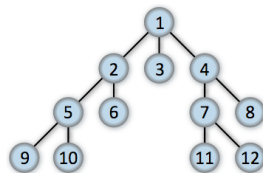
Add new cells to the graph. This is just a convenience method that wraps the [addCell](#) method.

dia.Graph.prototype.bfs

```
graph.bfs(element, iteratee [, opt])
```

Traverse the graph using the [Breadth-first search algorithm](#) starting at `element` (note the element itself will be visited too). `iteratee` is a function of the form `function(element, distance) {}` that will be called with the currently visited element and distance of that element from the root element of the search (the `element` passed to `bfs()`). If `iteratee` explicitly returns `false`, the search stops.

The following image shows the order in which elements are traversed in the graph:



Note that the `bfs()` algorithm is not only capable of traversing tree graphs but it can traverse any directed graph too.

It is smart enough not to traverse an element that was already visited.

If `opt.inbound` is `true`, reverse the search direction (it's like reversing all the link directions, i.e. swapping their `source` and `target`).

If `opt.outbound` is `true`, search follows the link directions. Calling `bfs()` with `opt.outbound` set to `true` is the most common case (graph is traversed following the direction of links).

If none of `opt.inbound` and `opt.outbound` are used or both options are set to `true`, the graph is traversed in both directions (very rare use case).

If `opt.deep` is `true`, the traversal takes into account embedded elements too. This option has the usual meaning as in other methods where `deep` option is used. For example, in a hierarchy A (top level element), A1 (embedded in A), B (top level element), where A is not directly connected to B but its embedded element is (there is a link from A1 ----> B), `bfs(A)` would not visit B but `bfs(A, function() {}, { deep: true })` would.

dia.Graph.prototype.clear

```
graph.clear([options])
```

Remove all the cells from the graph. `options` object can optionally contain additional data that is passed over to the event listeners of the graph cells remove event.

dia.Graph.prototype.cloneCells

```
graph.cloneCells(cells)
```

Clone all the cells (elements and/or links) from the `cells` array and return an object that maps the original cell ID to the clone (i.e. an object of the form `{ [original cell ID]: [clone] }`). The reason why this object is returned instead of an array of clones is that it is very useful to know which object the clone was created for.

The number of clones returned equals `cells.length`. This function does not simply clone all the cells but it also reconstructs all the source/target and parent/embed references within `cells`. This is very useful. For example, for a graph `A --- L ---> B`, `cloneCells([A, L, B])` returns `{ A.id: A2, L.id: L2, B.id: B2 }` resulting in a graph `A2 --- L2 ---> B2`, i.e. the source and target of the link `L2` is changed to point to `A2` and `B2` (in contrast to just looping over `cells` and calling `cell.clone()` on each item).

dia.Graph.prototype.cloneSubgraph

```
graph.cloneSubgraph(cells [, opt])
```

Clone the whole subgraph, including all the connected links whose source/target is in the subgraph. This is equivalent to calling `graph.cloneCells(graph.getSubgraph(cells))`.

If `opt.deep` is `true`, take into account embedded cells of the subgraph cells.

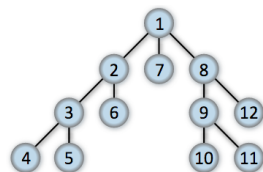
Return an object of the form `{ [original cell ID]: [clone] }`.

dia.Graph.prototype.dfs

```
graph.dfs(element, iteratee [, opt])
```

Traverse the graph using the [Depth-first search algorithm](#) starting at `element` (note the element itself will be visited too). `iteratee` is a function of the form `function(element, distance) {}` that will be called with the currently visited element and distance of that element from the root element of the search (the `element` passed to `dfs()`). If `iteratee` explicitly returns `false`, the search stops.

The following image shows the order in which elements are traversed in the graph:



Note that the `dfs()` algorithm is not only capable of traversing tree graphs but it can traverse any directed graph too. It is smart enough not to traverse an element that was already visited.

If `opt.inbound` is `true`, reverse the search direction (it's like reversing all the link directions, i.e. swapping their `source` and `target`).

If `opt.outbound` is `true`, search follows the link directions. Calling `dfs()` with `opt.outbound` set to `true` is the most common case (graph is traversed following the direction of links).

If none of `opt.inbound` and `opt.outbound` are used or both options are set to `true`, the graph is traversed in both directions (very rare use case).

If `opt.deep` is `true`, the traversal takes into account embedded elements too. This option has the usual meaning as in other methods where `deep` option is used. For example, in a hierarchy A (top level element), A1 (embedded in A), B (top level element), where A is not directly connected to B but its embedded element is (there is a link from A1 ----> B), `dfs(A)` would not visit B but `dfs(A, function() {}, { deep: true })` would.

dia.Graph.prototype.disconnectLinks

`graph.disconnectLinks(element)`

Disconnect all the associated links with the `element`.

dia.Graph.prototype.findModelsFromPoint

`graph.findModelsFromPoint(point)`

Find elements (instance of `joint.dia.Element`) under a certain point in the graph. `point` is an object with `x` and `y` properties. Returns an array of elements whose bounding box contains `point`. Note that there can be more than one element as elements might overlap.

dia.Graph.prototype.findModelsInArea

`graph.findModelsInArea(rect)`

Find elements (instance of `joint.dia.Element`) in a certain area in the graph. `rect` is an object with `x`, `y`, `width` and `height` properties. Returns an array of elements whose bounding box top/left coordinate falls into the `rect` rectangle.

dia.Graph.prototype.findModelsUnderElement

`graph.findModelsUnderElement(element [, opt])`

Find all the elements (instances of `joint.dia.Element`) that are located below `element`. `opt.searchBy` parameter optionally determines what it means for an element to be below another element. Possible values are `'bbox'` (default), `'center'`, `'origin'`, `'corner'`, `'topRight'`, and `'bottomLeft'`.

dia.Graph.prototype.fromJSON

`graph.fromJSON(json, [options])`

Load the graph from a JSON object (not string).

```
var jsonString = JSON.stringify(graph)
// ... send jsonString to the server,
// store it to the localStorage or do whatever you want
// ... later on
graph.fromJSON(JSON.parse(jsonString))
```

`options` object can optionally contain additional data that is passed over to the event listeners of the graph changes.

dia.Graph.prototype.getBBox

`graph.getBBox(cells[, opt])`

Returns the bounding box that surrounds all the given cells. Links are ignored. An example:

```
var bbox = graph.getBBox(graph.getElements());
// { x: Number, y: Number, width: Number, height: Number }
```

dia.Graph.prototype.getCell

`graph.getCell(id)`

Get a cell from the graph by its `id`.

dia.Graph.prototype.getCells

`graph.getCells()`

Get all the elements and links in the graph.

dia.Graph.prototype.getCommonAncestor

`graph.getCommonAncestor(cells...)`

Return the common ancestor of all the cells passed as arguments. For example, if an element `B` is embedded in an element `A` and an element `C` is also embedded in the element `A`, `graph.getCommonAncestor(B, C)` returns the element `A`. This also works on an arbitrary deep hierarchy.

dia.Graph.prototype.getConnectedLinks

`graph.getConnectedLinks(element, opt)`

Get all the associated links to the `element`. If `opt.inbound === true`, only inbound links will be returned. Similarly, if `opt.outbound === true`, only outbound links will be returned. If `opt.deep` is `true`, all the links that are connected to any of the descendant elements (embedded and deeply embedded) are returned.

dia.Graph.prototype.getElements

`graph.getElements()`

Get all the elements in the graph (i.e. omit links).

dia.Graph.prototype.getFirstCell

```
graph.getFirstCell()
```

Get the first cell (element or link) in the graph. The first cell is defined as the cell with the lowest `z` property (the cell most in the back, see the Presentation section of [joint.dia.Element](#)).

dia.Graph.prototype.getLastCell

```
graph.getLastCell()
```

Get the last cell (element or link) in the graph. The last cell is defined as the cell with the highest `z` property (the cell most in the front, see the Presentation section of [joint.dia.Element](#)).

dia.Graph.prototype.getLinks

```
graph.getLinks()
```

Get all the links in the graph (i.e. omit elements).

dia.Graph.prototype.getNeighbors

```
graph.getNeighbors(element [, opt])
```

Get all the neighbors of `element` in the graph. Neighbors are all the elements connected to `element` via either an inbound or an outbound link. If `opt.inbound` is `true`, only inbound neighbors (all neighbors connected with a link who's `target` is the `element`) will be returned. Similarly, if `opt.outbound` is `true`, only outbound neighbors will be returned. If `opt.deep` is `true`, return also all the neighbors of all the elements embedded inside `element`.

dia.Graph.prototype.getPredecessors

```
graph.getPredecessors(element [, opt])
```

Return an array of all the predecessors of `element`. By default, [Depth-first search algorithm](#) is used (important for the order of returned elements).

If `opt.breadthFirst` is set to `true`, use [Breadth-first search algorithm](#) instead.

If `opt.deep` is set to `true`, take into account embedded elements too (see [dfs\(\)](#) for details).

dia.Graph.prototype.getSinks

```
graph.getSinks()
```

Return an array of all the leafs of the graph. Time complexity: $O(|V|)$.

dia.Graph.prototype.getSources

```
graph.getSources()
```

Return an array of all the roots of the graph. Time complexity: $O(|V|)$.

dia.Graph.prototype.getSubgraph

```
graph.getSubgraph(cells [, opt])
```

Return an array of cells that result from finding elements/links that are connected to any of the cells in the `cells` array. This function loops over `cells` and if the current cell is a link, it collects its source/target elements; if it is an element, it collects its incoming and outgoing links if both the link ends (source/target) are in the `cells` array. For example, for a single element, the result is that very same element. For two elements connected with a link: `A --- L ---> B`, the result of `getSubgraph([A, B])` is `[A, L, B]` and the result of `getSubgraph([L])` is also `[A, L, B]`. If `opt.deep` is `true` take into account all the embedded cells too when finding neighboring links/elements.

dia.Graph.prototype.getSuccessors

```
graph.getSuccessors(element [, opt])
```

Return an array of all the successors of `element`. By default, [Depth-first search algorithm](#) is used (important for the order of returned elements).

If `opt.breadthFirst` is set to `true`, use [Breadth-first search algorithm](#) instead.

If `opt.deep` is set to `true`, take into account embedded elements too (see [dfs\(\)](#) for details).

dia.Graph.prototype.isNeighbor

```
graph.isNeighbor(elementA, elementB [, opt])
```

Return `true` if `elementB` is a neighbor of `elementA`. If `opt.deep` is set to `true`, take into account embedded elements.

If `opt.outbound` is set to `true`, return `true` only if `elementB` is a succeeding neighbor of `elementA`.

Similarly, if `opt.inbound` is set to `true`, return `true` only if `elementB` is a preceding neighbor of `elementA`.

dia.Graph.prototype.isPredecessor

```
graph.isPredecessor(elementA, elementB)
```

Return `true` if `elementB` is a predecessor of `elementA`.

dia.Graph.prototype.isSink

```
graph.isSink(element)
```

Return `true` if `element` is a leaf, i.e. there is no link coming out of the element. Time complexity: `O(1)`.

dia.Graph.prototype.isSource

```
graph.isSource(element)
```

Return `true` if `element` is a root, i.e. there is no link that targets the element. Time complexity: `O(1)`.

dia.Graph.prototype.isSuccessor

```
graph.isSuccessor(elementA, elementB)
```

Return `true` if `elementB` is a successor of `elementA`.

dia.Graph.prototype.removeCells

```
graph.removeCells(cells[, opt])
```

```
graph.removeCells(cell, cell, ..[, opt])
```

Removes the given cells from the graph.

dia.Graph.prototype.removeLinks

```
graph.removeLinks(element)
```

Remove all the associated links with the `element`.

dia.Graph.prototype.resetCells

```
graph.resetCells(cells[, opt])
```

```
graph.resetCells(cell, cell, ..[, opt])
```

Reset cells in the graph. Update all the cells in the graph in one bulk. This is a more efficient method of adding cells to the graph if you ou want to replace all the cells in one go. `options` object can optionally contain additional data that is passed over to the event listeners of the graph reset event.

dia.Graph.prototype.search

```
graph.search(element, iteratee [, opt])
```

Traverse the graph starting at `element` following links. This function is a wrapper around `dfs()` or `bfs()`. By default, it uses `dfs()`. If `opt.breadthFirst` is set to `true`, `bfs()` will be used instead.

dia.Graph.prototype.toJSON

```
graph.toJSON()
```

Return an object representing the graph ready for JSON serialization. This can be used for persistence or serialization. Note that this method doesn't return a JSON string but rather an object that can be then serialized to JSON with `JSON.stringify()`.

dia.Graph.prototype.translate

```
graph.translate(tx, ty [, opt])
```

Translate all cells in the graph by `tx` and `ty` pixels. It uses the `dia.Element.translate()` and `dia.Link.translate()` methods internally.

dia.Link

The basic model for diagram links. It inherits from `joint.dia.Cell` with a few additional properties and methods specific to links.

For links there are two crucial attributes: `source` and `target`. It defines the starting point and the end point of the link. It can be defined as follows:

```
new joint.dia.Link({
  source: { id: 'source-id' },
  target: { id: 'target-id', port: 'port_id' }
});

new joint.dia.Link({
  source: { id: 'source-id' },
  target: { x: 100, y: 100 }
});
```

dia.Link.events

- `change` - generic event triggered for any change on the link
- `change:source` - triggered when the link changes its source
- `change:target` - triggered when the link changes its target
- `change:attrs` - triggered when the link changes its attributes
- `change:smooth` - triggered when the link toggled interpolation
- `change:manhattan` - triggered when the link toggled orthogonal routing
- `change:vertices` - triggered when the link changes its vertices array
- `change:z` - triggered when the link is moved in the z-level (`toFront` and `toBack`)
- `transition:start` - triggered when a transition starts.
- `transition:end` - triggered when a transition ends.

```
link.on('change:source', function() { alert('source of the link changed') })
```

dia.Link.prototype.addTo

`link.addTo(graph)`

Add the link to the `graph` (an instance of `joint.dia.Graph`). This is equivalent to calling `graph.addCell(link)`.

dia.Link.prototype.attr

`link.attr(attrs)`

Set SVG attributes on subelements. This is a method analogous to `attr` method of `Joint.dia.Element`. The keys of the `attrs` object are CSS selectors matching the SVG element the link consists of. The values are objects containing SVG attributes and their values. `attrs` object will be mixed with `attrs` property of the `link` model. This is a convenient way of rewriting only some of the attributes of the SVG elements. For overwriting all attributes of all SVG elements, use `link.set('attrs', attrs)`. Here it is important to mention how the markup of a link looks like:

```
<path class="connection"/>
<path class="marker-source"/>
<path class="marker-target"/>
<path class="connection-wrap"/>
<g class="labels" />
<g class="marker-vertices"/>
<g class="marker-arrowheads"/>
<g class="link-tools" />
```

As you can see, the link consists of a couple of SVG path elements and couple of SVG group elements:

- `.connection` is the actual line of the link.
- `.connection-wrap` is an SVG path element that covers the `.connection` element and is usually thicker so that the link is able to handle pointer events (mousedown, mousemove, mouseup) that didn't target the thin `.connection` path exactly. This makes it easy to "grab" the link even though the mouse cursor didn't point exactly at the (usually thin) `.connection` path element.
- `.marker-source` and `.marker-target` are the arrowheads of the link.

```
link.attr({
  '.connection': { stroke: 'blue' },
  '.marker-source': { fill: 'red', d: 'M 10 0 L 0 5 L 10 10 z' },
  '.marker-target': { fill: 'yellow', d: 'M 10 0 L 0 5 L 10 10 z' }
});
```

An alternative call using a string path and a value:

```
link.attr('.marker-source/fill', 'green');
```

dia.Link.prototype.clone

```
link.clone()
```

Returns a new instance of the link with identical attributes.

dia.Link.prototype.disconnect

```
link.disconnect()
```

Disconnect the link from its `source` and `target` elements. The `source` and `target` then become a point at `[0,0]`.

dia.Link.prototype.findView

```
link.findView(paper)
```

Find view (`joint.dia.LinkView`) for the link model in the `paper`. This is a shortcut to the equivalent call `paper.findViewByModel(link)`

dia.Link.prototype.getAncestors

```
link.getAncestors()
```

Return an array of all the ancestors of this link starting from the immediate parent all the way up to the most distant ancestor.

dia.Link.prototype.getSourceElement

```
link.getSourceElement()
```

Return the source element of the link or `null` if there is none.

dia.Link.prototype.getTargetElement

```
link.getTargetElement()
```

Return the target element of the link or `null` if there is none.

dia.Link.prototype.getTransitions

```
link.getTransitions()
```

Return an array of all active transitions (their paths).

dia.Link.prototype.hasLoop

```
link.hasLoop([opt])
```

Return `true` if this link is a "loop link". In a "loop link" `source` and `target` are equal. If `opt.deep` is `true`, the notion of a "loop link" is extended to a deep hierarchy. For example, if the link connects a parent element with one of its embedded elements, then the link is considered a "loop link" too.

dia.Link.prototype.isElement

```
link.isElement()
```

Always returns `false`. The reason the method is here is that both `joint.dia.Element` and `joint.dia.Link` inherit from `joint.dia.Cell`. This method is useful if you don't know what the cell is. Calling `el.Element()` is equivalent to `el instanceof joint.dia.Element`. Example:

```
var cell = graph.getCell(myId)
if (cell.isElement()) {
  // Do something if the cell is an element.
}
```

dia.Link.prototype.isEmbeddedIn

```
link.isEmbeddedIn(element [, opt])
```

Return `true` if the link is embedded in an element `element`. If `opt.deep` is `false`, only direct parentage will be checked. `opt.deep` is `true` by default.

dia.Link.prototype.isLink

```
link.isLink()
```

Always returns `true`. The reason the method is here is that both `joint.dia.Element` and `joint.dia.Link` inherit from `joint.dia.Cell`. This method is useful if you don't know what the cell is. Calling `el.isLink()` is equivalent to `el instanceof joint.dia.Link`. Example:

```
var cell = graph.getCell(myId)
if (cell.isLink()) {
  // Do something if the cell is a link.
}
```

dia.Link.prototype.label

`link.label(index, properties, opt)`

Set `properties` on a label indexed by `index`. A link can have multiple labels each having different position and attributes.

```
{
  markup: <string>,
  position: <number>,
  attrs: { <selector>: <SVG attributes | JointJS attributes> }
}
```

Where `position` is the position of the label with regard to the link `.connection` SVG path. It's supposed to be an object with `distance` and `offset` properties. Where `distance` defined as below.

- If the `distance` is in `[0,1]` interval then the position of the label is considered to be defined as a percentage of the total length of the `.connection` path. E.g. distance `0.5` puts the label to the middle of the link.
- If the `distance` is `>1` then the label will be put `distance` pixels starting from the beginning of the `.connection` path.
- If the `distance` is `<0` puts the label `Math.abs(distance)` pixels starting from the end of the `.connection` path.

And `offset` defined as below.

- If the `offset` is defined as a number the position of the label is moved to the left (`<0`) or to the right (`>0`) perpendicular in the direction of the path.
- If the `offset` is defined as an object with `x` and `y` properties. It moves the label by `x` and `y` in the paper local coordinate system.

Alternatively the `position` could be defined as a number that behaves as `distance` defined above.

`attrs` is again an object with CSS selectors as keys and SVG attributes as values. The default label markup is defined as follows:

```
<g class="label"><rect /><text /></g>
```

Where `<rect>` is a box below the label.

```
link.label(0, {
  position: .5,
  attrs: {
    rect: { fill: 'white' },
    text: { fill: 'blue', text: 'my label' }
  }
});
```

Note that all labels are stored in an array on the link model under the attribute `labels`.

```
link.prop('labels'); // returns an array of label properties
```

dia.Link.prototype.presentation

The shape of the link is determined by the `vertices`, `connector` and `router` properties.

The `vertices` array contains a list of points which are the points which the link should cross.

```
link.get('vertices')
link.set('vertices', [{ x: 100, y: 120 }, { x: 150, y: 60 }])
```

A `router` takes an array of vertices defined on the model and transforms them to an array of points (**route**) that the link should go through. The difference between vertices and the route is that vertices are user-defined whilst route is computed.

There are four built-in routers (manhattan, metro, orthogonal and oneSide) to your disposal. The first two are so called "smart routers" and they automatically avoid elements that are in their way.

The `orthogonal`, `manhattan` and `oneSide` routers generate points that can be connected by vertical and horizontal line segments only. The `metro` router generates points that can be connected also with diagonal line segments.

```
link.set('router', { name: 'manhattan' });
link.set('router', { name: 'oneSide' });
link.set('router', { name: 'metro' });
link.set('router', { name: 'orthogonal' });
```

To keep backwards compatible it's possible to set the `manhattan` property to `true` in order to enable the orthogonal routing.

```
// old approach
link.set('manhattan', true)
```

The `manhattan` router has some additional useful options that determine how the algorithm behaves. These options can be passed into the `args` property and are:

- `excludeTypes` - an array of element types that should not be considered obstacles
- `excludeEnds` - either `'source'` or `'target'` string that tells the algorithm that the element at that specified end of the link should not be considered an obstacle
- `startDirections` - an array of sides of the link source element that the link can start from. The default is all the sides `['left', 'right', 'top', 'bottom']`. Change this in situations where you want the link to e.g. always start at the bottom of the source element (the value would be `['bottom']`)
- `endDirections` - an array of sides of the link target element that the link can stick to. The default is all the sides `['left', 'right', 'top', 'bottom']`

Example:

```
link.set('router', {
  name: 'manhattan',
  args: {
    startDirections: ['top'],
    endDirections: ['bottom'],
    excludeTypes: ['myNamespace.MyCommentElement']
  }
});
```

Complete list of options for `manhattan`:

startDirection	Array<string>	Possible starting directions from an element. Default is <code>['left', 'right', 'top', 'bottom']</code> .
endDirection	Array<string>	Possible ending directions to an element. Default is <code>['left', 'right',</code>

		<code>'top', 'bottom']</code> .
excludeTypes	<i>Array<string></i>	Should be any element with a certain type not to be consider as an obstacle.
step	<i>number</i>	Size of the step to find a route. Lower number -> higher complexity. <code>manhattan</code> performs the best when <code>`step`</code> has the same value as <code>dia.Paper.option.gridSize</code> .
perpendicular	<i>boolean</i>	Use of the perpendicular linkView option to connect center of element with first vertex.
maximumLoops	<i>number</i>	If number of route finding loops exceed the maximum, stops searching and returns fallback route. Higher number -> higher complexity.
excludeEnds	<i>Array<string></i>	Should be source or target not to be consider as an obstacle.
maxAllowedDirectionChange	<i>number</i>	Maximum change of the direction.

The `oneSide` routes the link in a given direction, creating exactly three orthogonal segments. It accepts the following options.

- `side` - the direction of the route. Either `'left'`, `'right'`, `'top'` or `'bottom'` (default).
- `padding` - a minimal gap between a connection point and the first/last vertex. It defaults to `40`.

Example:

```
link.set('router', {
  name: oneSide,
  args: {
    side: 'top',
    padding: 30
  }
});
```

New routers can be defined in the `joint.routers.[name of your router]` namespace or passed directly as functions to the `router` property of your links. The router is a function of the form `function(vertices, args, linkView)` and must return an array of vertices that the link should go through.

A `connector` takes points returned by a router and generate SVG path commands so the link can be rendered. There are four connectors (normal, smooth, rounded and jumpover) to your disposal.

The `normal` connector connects points with straight lines. The `rounded` connector does the same but it smooths all the edges. The `smooth` connector interpolates the points using a cubic bezier curve.

```
link.set('connector', { name: 'normal' });
link.set('connector', { name: 'smooth' });
```

The curve of edges can be specified by `radius` argument passed to the `rounded` connector.

```
link.set('connector', { name: 'rounded', args: { radius: 10 } });
```

The `jumpover` connector draws straight lines with small arcs at positions, where a link crosses an another. It accepts the following options.

- `size` - the size of the jump. It defaults to `5`.

- `jump` - the style of the jump. Either `'arc'` (default), `'cubic'` (similar to arc, but uses normalized path data commands only) or `'gap'`.

```
link.set('connector', { name: 'jumpover', args: { type: 'gap' } });
```

To keep backwards compatible it's possible to set the `smooth` property to `true` in order to enable smooth connector.

```
// old approach
link.set('smooth', true)
```

Note that code responsible for routing and rendering links is completely pluggable so the family of routers and connectors can be easily extended.

New connectors can be defined (similarly to routers) in the `joint.connectors.[name of your connector]` namespace or passed directly as functions to the `connector` property of your links. The connector is a function of the form `function(sourcePoint, targetPoint, vertices, args, linkView)` and must return a string representing the [SVG path data](#) that will be used to render the link.

Styling of the link is contained in the `attrs` property which has exactly the same structure as the `attrs` property of elements. Please refer to the [joint.dia.Element](#) section and [joint.dia.Link](#) sections of this page for more information on this.

Links also have the `z` property which is the z-level of the link. `z` property has exactly the same meaning as the `z` property of elements.

dia.Link.prototype.prop

`link.prop(properties)`

Set properties, possibly nested, on the element model. This is an equivalent of the `attr()` method but this time for custom data properties.

```
link.prop('name/first', 'John')
link.prop('name/first') // 'John'
link.prop({ name: { first: 'John' } })
// Nested arrays are supported too:
link.prop('mylist/0/data/0/value', 50)
link.prop({ mylist: [ { data: [ { value: 50 } ] } ] })
```

As you can see, this is the exact same method as the [joint.dia.Element:prop\(\)](#).

dia.Link.prototype.remove

`link.remove()`

Remove the link from the graph.

dia.Link.prototype.removeAttr

`link.removeAttr(path, [options])`

Remove a previously set attribute from the link. `path` can either be a string that specifies the path to the, possibly nested, attribute to be removed or an array of more paths. The associated link view makes sure the link gets re-rendered properly. If `options` is passed, it can contain data that is passed over to the event listeners for the `change:attrs` event triggered on the link itself and also on the graph the link is in.

dia.Link.prototype.reparent

```
link.reparent()
```

Automatically find and set the best parent element for the link so that when the parent element is moved, the link and all its vertices are moved too. The best parent is determined as the [common ancestor](#) of the source and target elements of the link. Useful for hierarchical diagrams. See the [DEVS demo](#) on how this can be used.

dia.Link.prototype.scale

```
link.scale(sx, sy, origin[, opt])
```

Scales the link's points (vertices) relative to the given origin.

dia.Link.prototype.stopTransitions

```
link.stopTransitions([path])
```

Stops all running transitions. If parameter `path` is provided, it will stop only transitions specified by this path.

dia.Link.prototype.toBack

```
link.toBack()
```

Move the link so it is behind all other cells (elements/links). This is a method analogous to [toBack](#) method of `Joint.dia.Element`.

dia.Link.prototype.toFront

```
link.toFront()
```

Move the element so it is on top of all other cells (elements/links). This is a method analogous to [toFront](#) method of `Joint.dia.Element`.

dia.Link.prototype.toJSON

```
link.toJSON()
```

Return a copy of the link's attributes for JSON serialization. This is a method analogous to [toJSON](#) method of `Joint.dia.Element`.

dia.Link.prototype.transition

```
link.transition(path, value [, opt])
```

Allows to change the link's property gradually over a period of time. This is a method analogous to [transition](#) method of `Joint.dia.Element`.

```
link.transition('target', { x: 250, y: 250 }, {
  delay: 100,
  duration: 500,
  timingFunction: joint.util.timing.bounce,
  valueFunction: joint.util.interpolate.object
});
// will start changing the link target coordinates in 100ms, for period of 500ms a
```

dia.Link.prototype.translate

```
link.translate(tx, ty, [options])
```

Translate the link vertices (and source and target if they are points) by `tx` pixels in x axis and `ty` pixels in y axis. If `options` object is passed, it can contain data that is passed over the the event listeners for the change event on the link or graph.

dia.LinkView

The view for the [joint.dia.Link](#) model. It inherits from [joint.dia.CellView](#) and is responsible for:

- Rendering an link inside of a paper
- Handling the link's pointer events
- Provides various methods for working with the link (visually)

To find the view associated with a specific link (model), use the [findViewByModel](#) method of the paper.

```
var linkView = paper.findViewByModel(link);
```

Custom LinkView

It is possible to use a custom link view for all your links in a paper. This can be set up via the `linkView` option on the [paper object](#). There are a couple of options that you can use when you inherit from `joint.dia.LinkView` and decide to use your own link view instead. These options are:

- **shortLinkLength** - when link gets shorter than the length specified, the link view will automatically scale down the link tools by half. It defaults to `40`.
- **doubleLinkTools** - sometimes when links overlap each other, the link tools stack one over the other making it impossible to reach the ones below the top one. By setting `doubleLinkTools` to `true`, you can force the link view to create a copy of the link tools and position it on the other end of the link. Note that this happens only if the link gets longer than `longLinkLength`.
- **longLinkLength** - when link gets longer than the length specified and `doubleLinkTools` is set to `true`, a copy of the link tools will be rendered on the other end of the link.
- **linkToolsOffset** - the offset of the link tools from the beginning of the link. It defaults to `40`.
- **doubleLinkToolsOffset** - the offset of the copy of the link tools from the end of the link. It defaults to `60`. This makes only sense if `doubleLinkTools` is set to `true`.

Example:

```
var paper = new joint.dia.Paper({
  linkView: joint.dia.LinkView.extend({
    options: _.defaults({
      doubleLinkTools: true
    }, joint.dia.LinkView.prototype.options)
  })
});
```

dia.LinkView.prototype.addVertex

Add a new vertex (an object with `x` and `y` properties) to the link. Normally, vertices are set through the `vertices` property of link models. However, sometimes it is useful to add a new vertex to the link and let the link determine to what index such a vertex should be added in the `vertices` array. For that, it must be the link view that does that. The link view tries to add the vertex to different indices in the `vertices` array, renders the connection SVG path and checks if the path length changed "significantly". If it did, it tries another index. If not, the found index is most likely to be the one we want our new vertex to be added to. For instance, let's say you want to add a new vertex on double click instead of the default click event. You can create your paper in the following way:

```
var paper = new joint.dia.Paper({
  // ...
  linkView: joint.dia.LinkView.extend({
    pointerdblclick: function(evt, x, y) {
      if (V(evt.target).hasClass('connection') || V(evt.target).hasClass('connection')) {
        this.addVertex({ x: x, y: y });
      }
    }
  }),
  interactive: function(cellView) {
    if (cellView.model instanceof joint.dia.Link) {
      // Disable the default vertex add functionality on pointerdown.
      return { vertexAdd: false };
    }
    return true;
  }
  // ...
});
```

dia.LinkView.prototype.getBBox

`linkView.getBBox()`

Return a bounding box for a link view.

dia.LinkView.prototype.getConnectionLength

`linkView.getConnectionLength()`

Return a total length of a link in pixels.

dia.LinkView.prototype.sendToken

`linkView.sendToken(token, [options], [callback])`

Send a token along the link. `token` is an SVG element (or Vectorizer element) that will be animated along the link path for `options.duration` milliseconds (default is 1000ms). `callback` will be called once the token reaches the end of the link path. `options.direction` specifies whether the animation should play forwards (`'normal'` - from the link source to target, which is the default) or backwards (`'reverse'`).

```
// Send an SVG circle token along the link.
var vCircle = V('circle', { r: 7, fill: 'green' });
link.findView(paper).sendToken(vCircle, { duration: 500, direction: 'reverse' }, function() {
  console.log('animation end');
});
```

Note that in the code above, we use the Vectorizer mini-library to create the SVG circle element.

Please see the [Petri Net simulator demo](https://resources.jointjs.com/docs/jointjs/v2.0/joint.html#shapes.devs) for a full working example.

dia.Paper.constructor

joint.dia.Paper is the view for the `joint.dia.Graph` model. It inherits from the [Backbone View](#). When a paper is associated with a graph, the paper makes sure that all the cells added to the graph are automatically rendered.

```
var graph = new joint.dia.Graph
var paper = new joint.dia.Paper({
  el: $('#paper'),
  width: 600,
  height: 400,
  gridSize: 10,
  model: graph
});

var rect = new joint.shapes.basic.Rect({
  position: { x: 50, y: 70 },
  size: { width: 100, height: 40 }
});

graph.addCell(rect);
```

Paper automatically handles this change and renders the rectangle to the SVG document that it internally holds.

dia.Paper.events

The following list contains events that you can react on in the paper:

- `cell:pointerdown` - triggered when a pointer is pressed on a cell. Takes `cellView`, `evt`, `x` and `y` as arguments.
- `cell:pointermove` - triggered when a pointer is moved on a paper and `cell:pointerdown` was previously handled. Takes `cellView`, `evt`, `x` and `y` as arguments.
- `cell:pointerup` - triggered when a pointer is released on a paper and `cell:pointerdown` was previously handled. Takes `cellView` and `evt` as arguments.
- `cell:pointerdblclick` - triggered when the user double-clicks a cell. Takes `cellView`, `evt`, `x` and `y` as arguments.
- `cell:pointerclick` - triggered when the user clicks a cell. Takes `cellView`, `evt`, `x` and `y` as arguments.
- `cell:mouseover` - triggered when the user enters the cellView or its child element. Takes `cellView` and `evt` as arguments.
- `cell:mouseout` - triggered when the user leaves the cellView or its child element. Takes `cellView` and `evt` as arguments.
- `cell:mouseenter` - triggered when the user enters the cell view area. Takes `cellView` and `evt` as arguments.
- `cell:mouseleave` - triggered when the user leaves the cell view area. Takes `cellView` and `evt` as arguments.
- `link:mouseenter` - triggered when the user enters the link view area. Takes `cellView` and `evt` as arguments.
- `link:mouseleave` - triggered when the user leaves the link view area. Takes `cellView` and `evt` as arguments.
- `element:mouseenter` - triggered when the user enters the element view area. Takes `cellView` and `evt` as arguments.
- `element:mouseleave` - triggered when the user leaves the element view area. Takes `cellView` and `evt` as arguments.
- `blank:pointerdown` - triggered when a pointer is pressed on a blank area on the paper. Takes `evt`, `x` and `y` as arguments.
- `blank:pointerdblclick` - triggered when the user double-clicks a blank area on the paper. Takes `evt`, `x` and `y` as arguments.
- `blank:pointerclick` - triggered when the user clicks a blank area on the paper. Takes `evt`, `x` and `y` as arguments.

- `cell:mousewheel` - triggered when the user turns their mousewheel while the cursor is over a cell in the paper. Takes `cellView`, `evt`, `x`, `y`, `delta` as arguments.
- `blank:mousewheel` - triggered when the user turns their mousewheel while the cursor is over the blank area of the paper. Takes `evt`, `x`, `y`, `delta` as arguments.
- `cell:contextmenu` - triggered when the user right-clicks a cell in the paper. Takes `cellView`, `evt`, `x` and `y` as arguments.
- `blank:contextmenu` - triggered when the user right-clicks a blank area in the paper. Takes `evt`, `x` and `y` as arguments.
- `render:done` - triggered when the paper has finished rendering all the cell views in case `async` rendering is enabled.
- `cell:highlight` - triggered when `highlight()` method is called on either an element or a link. Note that this method is also called automatically when the user is reconnecting a link and the connection is valid (`validateConnection()` returns `true`) or if `embeddingMode` is enabled on the paper and the dragging element is above another element it could be dropped into (`validateEmbedding()` returns `true`). The handler for this event has the following signature: `function(cellView, el)`. The handler defaults to `function(cellView, el) { V(el).addClass('highlighted') }`. In other words, the `'highlighted'` CSS class is added and so you can style the highlighted element in CSS. If you want to use a different method for highlighting cells, call `paper.off('cell:highlight')` first to unregister the default handler and then `paper.on('cell:highlight', myHandler)` to register your own.
- `cell:unhighlight` - triggered when `unhighlight()` method is called on either an element or a link. See above the comments for `cell:highlight` event for further details.
- `link:connect` - triggered when a link is connected to a cell. Takes `linkView`, `evt`, `elementViewConnected`, `magnet`, `arrowhead` as arguments.
- `link:disconnect` - triggered when a link is disconnected from a cell. Takes `linkView`, `evt`, `elementViewDisconnected`, `magnet`, `arrowhead` as arguments.

```
paper.on('blank:pointerdown', function(evt, x, y) {
    alert('pointerdown on a blank area in the paper.')
})
```

dia.Paper.prototype.cancelRenderViews

```
paper.cancelRenderViews()
```

When the method is called any asynchronous rendering in progress is canceled.

dia.Paper.prototype.clearGrid

```
paper.clearGrid()
```

Hide the current grid.

dia.Paper.prototype.clientOffset

```
paper.clientOffset()
```

Returns coordinates of the paper viewport, relative to the application's client area.

dia.Paper.prototype.clientToLocalPoint

```
paper.clientToLocalPoint(p)
```

Transform client coordinates represented by point `p` to the paper local coordinates. This is especially useful when you have a mouse event object and want coordinates inside the paper that correspond to event `clientX` and `clientY` point.

```
var localPoint1 = paper.clientToLocalPoint({ x: evt.clientX, y: evt.clientY });
// alternative method signature
var localPoint2 = paper.clientToLocalPoint(evt.clientX, evt.clientY);
```

dia.Paper.prototype.clientToLocalRect

`paper.clientToLocalRect(rect)`

Transform the rectangle `rect` defined in the client coordinate system to the local coordinate system.

```
var bcr = paper.svg.getBoundingClientRect();
var localRect1 = paper.clientToLocalRect({ x: bcr.left, y: bcr.top, width: bcr.width, height: bcr.height });
// alternative method signature
var localRect2 = paper.clientToLocalRect(bcr.left, bcr.top, bcr.width, bcr.height);
// Move the element to the center of the paper viewport.
var localCenter = localRect1.center();
var elSize = element.size();
element.position(localCenter.x - elSize.width, localCenter.y - elSize.height);
```

dia.Paper.prototype.defineFilter

`paper.defineFilter(filterDefinition)`

Define an SVG filter for later reuse within the paper. The method returns the filter id and the `filterDefinition` must have the following form:

```
{
  name: <name of the filter>,
  args: <filter arguments>
}
```

Where `name` is the name of the filter. See below for the list of built-in filters. `args` is an object containing filter parameters. These parameters are dependent on the filter used and are described in the list below as well. Example usage:

```
var filterId = paper.defineFilter({
  name: 'dropShadow',
  args: {
    dx: 2,
    dy: 2,
    blur: 3
  }
});

svgNode.setAttribute('filter', 'url(#' + filterId + ');');
```

The following is the list of built-in filters. All these filters are defined in the `joint.util.filter` namespace. This namespace can be extended simply by adding a new method to it with one argument, an object with filter parameters, returning a string representing the SVG filter definition.

- `blur`
 - `x` - horizontal blur
 - `y` - vertical blur [optional, if not defined `y` is the same as `x`]
- `dropShadow`
 - `dx` - horizontal shift
 - `dy` - vertical shift
 - `blur` - blur
 - `color` - color
 - `opacity` - opacity
- `grayscale`

- o `amount` - the proportion of the conversion. `1` is completely grayscale. `0` leaves the element unchanged.
- `sepia`
 - o `amount` - the proportion of the conversion. `1` is completely sepia. `0` leaves the element unchanged.
- `saturate`
 - o `amount` - the proportion of the conversion. `0` is completely un-saturated. `1` leaves the element unchanged.
- `hueRotate`
 - o `angle` - the number of degrees around the color circle the input samples will be adjusted
- `invert`
 - o `amount` - the proportion of the conversion. `1` is completely inverted. `0` leaves the element unchanged.
- `brightness`
 - o `amount` - the proportion of the conversion. `0` makes the element completely black. `1` leaves the element unchanged.
- `contrast`
 - o `amount` - the proportion of the conversion. `0` makes the element completely black. `1` leaves the element unchanged.

`dia.Paper.prototype.defineGradient`

`paper.defineGradient`(gradientDefinition)

Define an SVG gradient for later reuse within the paper. The method returns the gradient id and the `gradientDefinition` must have the following form:

```
{
  type: <type of gradient>,
  stops: <stop colors>,
  attrs: <additional attributes>
}
```

Where `type` is either `'linearGradient'` or `'radialGradient'`, `attrs` is an object containing additional SVG attributes for the SVG gradient element and `stops` is an array of the ramps of color on the gradient. Each stop object is of the form:

```
{
  offset: <offset>,
  color: <color>,
  opacity: <opacity>
}
```

Where `offset` is a string representing the offset of the gradient stop, `color` indicates what color to use at that gradient stop and `opacity` is a number in the [0..1] range representing the transparency of the stop color. Example use:

```
var gradientId = paper.defineGradient({
  type: 'linearGradient',
  stops: [
    { offset: '0%', color: '#E67E22' },
    { offset: '20%', color: '#D35400' },
    { offset: '40%', color: '#E74C3C' },
    { offset: '60%', color: '#C0392B' },
    { offset: '80%', color: '#F39C12' }
  ]
});

svgNode.setAttribute('fill', 'url(#' + gradientId + ')');
```

For an introduction to gradients, please refer to the tutorial on [Filters and Gradients](#).

dia.Paper.prototype.defineMarker

`paper.defineMarker(markerDefinition)`

Define an SVG marker for later reuse within the paper. The method returns the marker id and the `markerDefinition` is an object with `type` property and any other visual attributes. The valid values for `type` are `'path'`, `'circle'`, `'ellipse'`, `'rect'`, `'polyline'` and `'polygon'`.

```

var markerId = paper.defineMarker({
  type: 'path', // SVG Path
  fill: '#666',
  stroke: '#333',
  // The coordinate system for defining the path data
  // -----
  // 0,0: marker-start, marker-end or marker-mid
  // n,0: n > 0 in path direction
  //      n < 0 opposite path direction
  // 0,m: m > 0 left to the path direction
  //      m < 0 right to the path direction
  d: 'M 10 -10 0 0 10 10 z'
});

// Draw an arrow at the start and the end of a path
svgPath.setAttribute('marker-start', 'url(#' + markerId + ')');
svgPath.setAttribute('marker-end', 'url(#' + markerId + ')');

```

dia.Paper.prototype.drawBackground

`paper.drawBackground(opt);`

Sets the paper background defined by the `opt` object. Please see the [background](#) paper option for available configuration.

dia.Paper.prototype.drawGrid

`paper.drawGrid([opt])`

Draw visual grid lines on the paper. Possible options:

- **color** - the color of the grid line (hex, RGB, etc).
- **thickness** - the thickness of the grid line (pixels).

dia.Paper.prototype.findView

`paper.findView(element)`

Find a view (instance of `joint.dia.ElementView` or `joint.dia.LinkView`) associated with a DOM element in the paper. `element` can either be a DOM element, jQuery object or a CSS selector. Sometimes, it is useful to find a view object for an element in the DOM. This method finds the closest view for any subelement of a view element.

dia.Paper.prototype.findViewByModel

`paper.findViewByModel(model)`

Find a view (instance of `joint.dia.ElementView` or `joint.dia.LinkView`) associated with a model. `model` can either be an instance of `joint.dia.Element` or `joint.dia.Link`.

dia.Paper.prototype.findViewsFromPoint

```
paper.findViewsFromPoint(point)
```

Find views (instance of `joint.dia.ElementView`) under a certain point in the paper. `point` is an object with `x` and `y` properties. Returns an array of views whose bounding box contains `point`. Note that there can be more than one view as views might overlap. Note there is a difference between this method and the `joint.dia.Graph.findModelsFromPoint`. A bounding box of a view can be different than the area computed by an element model `position` and `size`. For example, if a `<text>` SVG element in the shape is positioned relatively and shifted down below the normal shape area (e.g. using the [jointJS special attributes](#)), the bounding box of the view will be bigger than that of the model.

dia.Paper.prototype.findViewsInArea

```
paper.findViewsInArea(rect [, opt])
```

Find views (instance of `joint.dia.ElementView`) in a certain area in the paper. `rect` is an object with `x`, `y`, `width` and `height` properties. Return an array of views whose bounding box intersects the `rect` rectangle.

If `opt.strict` is `true`, return an array of views whose bounding box is contained within the `rect` rectangle (i.e. not only intersects it).

dia.Paper.prototype.fitToContent

```
paper.fitToContent([opt])
```

Expand/shrink the paper to fit the content inside it. Snap the resulting width/height to the grid defined by `opt.gridWidth` / `opt.gridHeight`. `opt.padding` adds to the resulting width/height of the paper. The `opt.padding` can either be a number in which case it represents the padding on all the sides or it can be an object of the form `{ top: Number, right: Number, bottom: Number, left: Number }`. By default the method fits the paper to a content with positive coordinates only and sets the origin to (0,0) for the resulting paper. To change this behaviour use the `allowNewOrigin` `['negative'|'positive'|'any']` option. Set `allowNewOrigin` to `'negative'` to account for content with negative coordinates. Set `allowNewOrigin` to `'positive'` to allow origin to be set to the top-left coordinate of the content. Set `allowNewOrigin` to `'any'` to apply both. This method might internally trigger the `"resize"` and `"translate"` events that can be handled by listening on the paper object (`paper.on('resize', myHandler)`). `opt.minWidth` and `opt.minHeight` define the minimum width and height of the paper and `opt.maxWidth` and `opt.maxHeight` define the maximum width and height of the paper after fitting it to the content.

You can try many of these options interactively in the [paper demo](#).

Deprecated usage: `paper.fitToContent(gridWidth, gridHeight, padding, opt)`

dia.Paper.prototype.getContentBBox

```
paper.getContentBBox()
```

Return the bounding box of the content inside the paper.

dia.Paper.prototype.isDefined

`paper.isDefined(graphicalObjectId)`

Return `true` if there is a graphical object (gradient, filter, marker) with `graphicalObjectId` already defined within the paper. Return `false` otherwise.

dia.Paper.prototype.localToClientPoint

`paper.localToClientPoint(p)`

Transform the point `p` defined in the local coordinate system to the client coordinate system.

```
var rightMidPoint = element.getBBox().rightMiddle();
var clientPoint1 = paper.localToClientPoint(rightMidPoint);
// alternative method signature
var clientPoint2 = paper.localToClientPoint(rightMidPoint.x, rightMidPoint.y);
// Draw an HTML rectangle next to the element.
var div = document.createElement('div');
div.style.position = 'fixed';
div.style.background = 'red';
div.style.left = clientPoint1.x + 'px';
div.style.top = clientPoint1.y + 'px';
div.style.width = '40px';
div.style.height = '40px';
div.style.marginLeft = '10px';
div.style.marginTop = '-20px';
document.body.appendChild(div);
```

dia.Paper.prototype.localToClientRect

`paper.localToClientRect(rect)`

Transform the rectangle `rect` defined in local coordinate system to the client coordinate system.

```
var bbox = element.getBBox();
var clientRect1 = paper.localToClientRect(bbox);
// alternative method signature
var clientRect2 = paper.localToClientRect(bbox.x, bbox.y, bbox.width, bbox.height);
// Draw an HTML rectangle above the element.
var div = document.createElement('div');
div.style.position = 'fixed';
div.style.background = 'red';
div.style.left = clientRect1.x + 'px';
div.style.top = clientRect1.y + 'px';
div.style.width = clientRect1.width + 'px';
div.style.height = clientRect1.height + 'px';
paper.el.appendChild(div);
```

dia.Paper.prototype.localToPagePoint

`paper.localToPagePoint(p)`

Transform the point `p` defined in the local coordinate system to the page coordinate system.

```
var rightMidPoint = element.getBBox().rightMiddle();
var pagePoint1 = paper.localToPagePoint(rightMidPoint);
// alternative method signature
```

```

var pagePoint2 = paper.localToPagePoint(rightMidPoint.x, rightMidPoint.y);
// Draw an HTML rectangle next to the element.
var div = document.createElement('div');
div.style.position = 'absolute';
div.style.background = 'red';
div.style.left = pagePoint1.x + 'px';
div.style.top = pagePoint1.y + 'px';
div.style.width = '40px';
div.style.height = '40px';
div.style.marginLeft = '10px';
div.style.marginTop = '-20px';
document.body.appendChild(div);

```

dia.Paper.prototype.localToPageRect

`paper.localToPageRect(rect)`

Transform the rectangle `rect` defined in local coordinate system to the page coordinate system.

```

var bbox = element.getBBox();
var pageRect1 = paper.localToPageRect(bbox);
// alternative method signature
var pageRect2 = paper.localToPageRect(bbox.x, bbox.y, bbox.width, bbox.height);
// Draw an HTML rectangle above the element.
var div = document.createElement('div');
div.style.position = 'absolute';
div.style.background = 'red';
div.style.left = pageRect1.x + 'px';
div.style.top = pageRect1.y + 'px';
div.style.width = pageRect1.width + 'px';
div.style.height = pageRect1.height + 'px';
document.body.appendChild(div);

```

dia.Paper.prototype.localToPaperPoint

`paper.localToPaperPoint(p)`

Transform the point `p` defined in the local coordinate system to the paper coordinate system.

```

var rightMidPoint = element.getBBox().rightMiddle();
var paperPoint1 = paper.localToPaperPoint(rightMidPoint);
// alternative method signature
var paperPoint2 = paper.localToPaperPoint(rightMidPoint.x, rightMidPoint.y);
// Draw an HTML rectangle next to the element.
var div = document.createElement('div');
div.style.position = 'absolute';
div.style.background = 'red';
div.style.left = paperPoint1.x + 'px';
div.style.top = paperPoint1.y + 'px';
div.style.width = '40px';
div.style.height = '40px';
div.style.marginLeft = '10px';
div.style.marginTop = '-20px';
paper.el.appendChild(div);

```

dia.Paper.prototype.localToPaperRect

`paper.localToPaperRect(rect)`

Transform the rectangle `rect` defined in the local coordinate system to the paper coordinate system.

```

var bbox = element.getBBox();
var paperRect1 = paper.localToPaperRect(bbox);
// alternative method signature
var paperRect2 = paper.localToPaperRect(bbox.x, bbox.y, bbox.width, bbox.height);
// Draw an HTML rectangle above the element.
var div = document.createElement('div');
div.style.position = 'absolute';
div.style.background = 'red';
div.style.left = paperRect1.x + 'px';
div.style.top = paperRect1.y + 'px';
div.style.width = paperRect1.width + 'px';
div.style.height = paperRect1.height + 'px';
paper.el.appendChild(div);

```

dia.Paper.prototype.matrix

`paper.matrix([SVGMatrix])`

When called with no parameter, the method returns the current transformation matrix (instance of [SVGMatrix](#)) of the paper. It sets the new viewport transformation based on the `SVGMatrix` otherwise.

```
paper.matrix({ a: 2, b: 0, c: 0, d: 2, e: 0, f: 0 }); // scale the paper twice
```

dia.Paper.prototype.pageOffset

`paper.pageOffset()`

Returns coordinates of the paper viewport, relative to the document.

dia.Paper.prototype.pageToLocalPoint

`paper.pageToLocalPoint(p)`

Transform the point `p` defined in the page coordinate system to the local coordinate system.

```

paper.on('blank:pointerup', function(evt) {
  var pagePoint = g.Point(evt.pageX, evt.pageY);
  var localPoint1 = this.pageToLocalPoint(pagePoint);
  // alternative method signature
  var localPoint2 = this.pageToLocalPoint(evt.pageX, evt.pageY);
  // Move the element to the point, where the user just clicks.
  element.position(localPoint1.x, localPoint1.y);
});

```

dia.Paper.prototype.pageToLocalRect

`paper.pageToLocalRect(rect)`

Transform the rectangle `rect` defined in the page coordinate system to the local coordinate system.

```

var x, y;
paper.on('blank:pointerdown', function(evt) {
  x = evt.pageX;
  y = evt.pageY;
});
paper.on('blank:pointerup', function(evt) {

```

```

var pageRect = g.Rect(x, y, evt.pageX - x, evt.pageY - y).normalize();
var localRect1 = this.paperToLocalRect(pageRect);
// alternative method signature
var localRect2 = this.paperToLocalRect(x, y, evt.pageX - x, evt.pageY - y).normalize();
// Move and resize the element to cover the area, that the user just selected.
element.position(localRect1.x, localRect1.y);
element.resize(localRect1.width, localRect1.height);
});

```

dia.Paper.prototype.paperToLocalPoint

paper.**paperToLocalPoint**(p)

Transform the point `p` defined in the paper coordinate system to the local coordinate system.

```

var paperCenter = g.Point(paper.options.width / 2, paper.options.height / 2);
var localPoint1 = paper.paperToLocalPoint(paperCenter);
// alternative method signature
var localPoint2 = paper.paperToLocalPoint(paper.options.width / 2, paper.options.height / 2);
// Move the element to the center of the paper viewport.
var elSize = element.size();
element.position(localPoint1.x - elSize.width, localPoint1.y - elSize.height);

```

dia.Paper.prototype.paperToLocalRect

paper.**paperToLocalRect**(rect)

Transform the rectangle `rect` defined in the paper coordinate system to the local coordinate system.

```

var paperRect = g.Rect(0, 0, paper.options.width, paper.options.height);
var localRect1 = paper.paperToLocalRect(paperRect);
// alternative method signature
var localRect2 = paper.paperToLocalRect(0, 0, paper.options.width, paper.options.height);
// Move and resize the element to cover the entire paper viewport.
element.position(localRect1.x, localRect1.y);
element.size(localRect1.width, localRect1.height);

```

dia.Paper.prototype.properties

The following list contains properties exposed by the paper object:

- `svg` - a reference to the SVG document object the paper uses to render all the graphics.
- `viewport` - a reference to the SVG group `<g>` element the paper wraps all the rendered elements and links into. Paper transformations such as scale is performed on this element.
- `defs` - a reference to the [SVG <defs> element](#) used to define SVG elements for later reuse. This is also a good place to put SVG masks, patterns, filters and gradients.

Normally, you do not need to access these properties directly but in some (advanced) situations, it is handy to have access to them.

dia.Paper.prototype.scale

paper.**scale**([sx, sy, ox, oy])

Scale a paper by `sx` factor in x axis and `sy` factor in y axis. `ox` and `oy` are optional and determine the origin of the scale transformation. This method effectively implements a paper zoom in/out. If the method is called a `"scale"` event is triggered on the paper. When the method is called with no parameter the current paper `scale` transformation is returned.

```
paper.scale(2) // scale 2x (uniformly)
paper.scale(2,3) // scale 2x in `x` axis 3x in `y` axis (non-uniformly)
paper.scale(2,2,100,100) // scale with the origin of the transformation at point `
paper.scale() // returns e.g. { sx: 2, sy: 2 }
```

dia.Paper.prototype.scaleContentToFit

`paper.scaleContentToFit([opt])`

Scale the paper content so that it fits the paper dimensions. If `opt.padding` is set (default is `0`), there will be an additional padding in the resulting, scaled, paper content. If `opt.preserveAspectRatio` is defined (default is `true`), the aspect ratio of the scaled paper content will be preserved. `opt.minScaleX`, `opt.minScaleY`, `opt.maxScaleX` and `opt.maxScaleY` can be optionally used to set the minimum and maximum allowed scale factor for both axis. `opt.scaleGrid` is a number that will be used to as a rounding factor for the resulting scale. For example, if the resulting scale factor is calculated to be `1.15` and your `opt.scaleGrid` is set to `0.2`, then the resulting scale factor will be rounded to `1.2`. Last option is `opt.fittingBBox` which can be an object of the form `{ x: [number], y: [number], width: [number], height: [number] }` and is the area of the paper that should be scaled. By default `opt.fittingBBox` is `{ x: 0, y: y, width: paper.options.width, height: paper.options.height }`, i.e. the whole paper area. If the method called a `"scale"` event can be triggered on the paper. To try it yourself see [paper demo](#).

dia.Paper.prototype.setDimensions

`paper.setDimensions(width, height)`

Change dimensions of a paper. Dimensions should always be passed to the options object of the `joint.dia.Paper` constructor. Use `setDimensions()` to change dimensions of the paper later on if needed. If the method is called a `"resize"` event is triggered on the paper.

dia.Paper.prototype.setGridSize

`paper.setGridSize(gridSize)`

Set the grid size of the paper.

dia.Paper.prototype.setInteractivity

`paper.setInteractivity(interactive)`

Set the interactivity of the paper. For example, to disable interactivity:

```
paper.setInteractivity(false);
```

dia.Paper.prototype.setOrigin

`paper.setOrigin(x, y)`

Deprecated in favor of `translate()`. Lets you modify the origin (zero coordinates) of a paper. An origin can also be passed to the options object of the `joint.dia.Paper` constructor. If the method is called a `"translate"` event is triggered on the paper.

dia.Paper.prototype.translate

`paper.translate(x, y)`

Lets you modify the origin (zero coordinates) of a paper. An origin can also be passed to the options object of the `joint.dia.Paper` constructor. If the method is called a `"translate"` event is triggered on the paper. If the method is called with no parameter the current paper `translate` transformation is returned.

```
paper.translate(100, 200) // set origin to `x=100` and `y=200`
paper.translate(100) // same as calling `translate(100,0)`
paper.translate() // returns e.g. { tx: 100, ty: 100 }
```

dia.Paper.prototype.options.async

`async` - when enabled, the paper renders cells added to the graph through `graph.resetCells()` or `graph.addCell()` asynchronously. This is very useful when you want to add a large number of cells into the graph. The rendering performance boost is significant and doesn't block the UI. The option accepts either `true` in which case default values are used or an object of the form `{ batchSize: <value> }` where you can specify the number of cells rendered in each animation frame (default is 50). Normally, the default `batchSize` works great but you might want to experiment with different values in case you encounter performance issues. It is important to note that when asynchronous rendering is used, some of the cell views might not yet be in the paper when you try to access them via the `paper.findViewByModel()`, `element.findView()` or `link.findView()` methods. What you should do in case you use async rendering is to wait for when the paper triggers the `render:done` event.

dia.Paper.prototype.options.background

An object defining the paper background color and image. It defaults to `false` meaning there is no background set. The configuration object can have the following properties.

- **color** property sets the paper background color. It accepts all the values accepted by the CSS `background-color` property. e.g. `'red'`, `'rgba(255, 255, 0, 0.5)'`, `'radial-gradient(ellipse at center, red, green);'`
- **image** property defines the path to the background image file. e.g. `'/my-background.png'`.
- **position** is an object `{ x: Number, y: Number }` defining the background image position. It also allows to use all the CSS `background-position` property values. In that case all the paper transformations have no impact on the background image position. It defaults to `center`.
- **size** is an object `{ width: Number, height: Number }` defining the background image size. It also allows to use all the CSS `background-size` property values. In that case all the paper transformations have no impact on the background size. It defaults to `auto auto`.
- **repeat** property defines how the background image is repeated. The value could be any value accepted by the CSS `background-repeat` property and few more defined by JointJS. Those are `flip-x`, `flip-y`, `flip-xy` and `watermark`. It defaults to `no-repeat`.
- **quality** is a coefficient specifying the quality of the image (e.g. `0.5` uses only 50% the image size for creating a pattern). Applicable only for the JointJS `repeat` option values. It defaults to `1`.
- **opacity** is a number in the range `[0,1]` specifying the transparency of the background image (`0` is fully transparent and `1` is fully opaque). It defaults to `1`.
- **watermarkAngle** is an angle in degrees specifying the slope of the watermark image. Applicable only for the `'watermark'` `repeat` option. It defaults to `20` deg.

```
background: {
  color: '#6764A7',
  image: 'jointjs-logo.png',
  repeat: 'watermark',
  opacity: 0.3
}
```

dia.Paper.prototype.options.cellViewNamespace

`cellViewNamespace` - this option can be used to change the default behavior of JointJS which by default reads cell view definitions from the `joint.shapes` namespace. For example, if a cell is of type `'myshapes.MyElement'`, then the paper looks up `joint.shapes.myshapes.MyElementView` object type when rendering the cell onto the screen. If `cellViewNamespace` is set, the paper will read the view definition from the `myShapesNamespace.myshapes.MyElementView` object instead. This is useful in situations where you - for any reason - don't want to use the `joint.shapes` namespace for defining your own custom shapes. This option is often used in combination with `cellNamespace` option on the `joint.dia.Graph` object.

dia.Paper.prototype.options.clickThreshold

`clickThreshold` - When number of mousemove events exceeds the `clickThreshold` there is no `pointerclick` event triggered after mouseup. It defaults to `0`.

dia.Paper.prototype.options.defaultConnector

`defaultConnector` - a connector that is used by links if no `connector` property is defined on them. It can be either an object or a function. See the [Presentation](#) section of `joint.dia.Link` for more information about connectors

dia.Paper.prototype.options.defaultLink

`defaultLink` - link that should be created when the user clicks and drags and active magnet (when creating a link from a port via the UI). Defaults to `new joint.dia.Link`. It can also be a function with signature `function(cellView, magnet) {}` that must return an object of type `joint.dia.Link`.

dia.Paper.prototype.options.defaultRouter

`defaultRouter` - a router that is used by links if no `router` property is defined on them. It can be either an object or a function. See the [Presentation](#) section of `joint.dia.Link` for more information about routers

dia.Paper.prototype.options.drawGrid

`drawGrid` - option whether the paper grid should be drawn or not. It could be a `boolean` or an `object`. It defaults to `false`. Define `color` and `thickness` to adjust the default grid pattern:

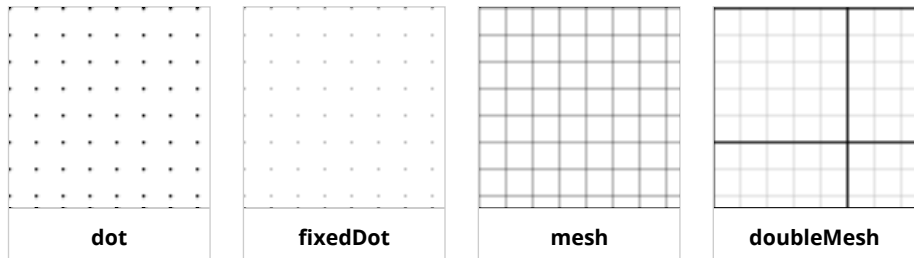
color	<i>string</i>	color of the default grid pattern.
thickness	<i>number</i>	thickness of the default grid pattern.

There are also some pre-defined grid patterns: `dot`, `fixedDot`, `mesh`, `doubleMesh`. If you'd like to use these patterns, define `drawGrid` options as follows:

name	<i>string</i>	name of the pre-defined pattern. Can be either <code>dot</code> , <code>fixedDot</code> , <code>mesh</code> , <code>doubleMesh</code>
args	<i>object</i> <i>array</i>	an extra arguments for the pre-defined grid patterns. It can be either an <code>object</code> (e.g. <code>dot</code> , <code>mesh</code>) or an <code>array</code> (e.g. <code>doubleMesh</code>)

Pre-defined grids with default settings:

The paper from the images below has been scaled 2 times and has `gridSize` set to 10.



```
drawGrid: true // default pattern (dot) with default settings

drawGrid: 'mesh' // pre-defined pattern with default settings
drawGrid: { name: 'mesh', args: { color: 'black' } }

drawGrid: {
  name: 'doubleMesh',
  args: [
    { color: 'red', thickness: 1 }, // settings for the primary mesh
    { color: 'green', scaleFactor: 5, thickness: 5 } // settings for the second
  ]
}
```

`dia.Paper.prototype.options.el`

`el` - CSS selector, jQuery object or a DOM element holding the container for the paper

`dia.Paper.prototype.options.elementView`

`elementView` - object that is responsible for rendering an element model into the paper. Defaults to `joint.dia.ElementView`. It can also be a function of the form `function(element)` that takes an element model and should return an object responsible for rendering that model onto the screen (in most cases a subtype of `joint.dia.ElementView`)

`dia.Paper.prototype.options.embeddingMode`

`embeddingMode` - when set to `true`, the paper is set to embedding mode. In this mode, when you drag an element and drop into another element, the element below becomes a parent of the dropped element (the dropped element gets automatically embedded into the parent element). Similarly, when you drag an element out of its parent, the element gets automatically unembedded from its parent. The embedding mode also makes sure all the connected links and child elements have proper `z` index set so that they stay visible. To control what elements can be embedded into what other elements, use the `validateEmbedding()` function on the paper (see below). This is useful for hierarchical diagrams. See the [DEVS demo](#) on how this can be used.

dia.Paper.prototype.options.gridSize

`gridSize` - size of the grid in pixels

dia.Paper.prototype.options.guard

`guard` - guard paper from handling a browser UI event. This function is of the form `function (evt, view)` and should return `true` if you want to prevent the paper from handling the event `evt`, `false` otherwise. This is an advanced option that can be useful if you have your own logic for handling events.

dia.Paper.prototype.options.height

`height` - height of the paper in pixels

dia.Paper.prototype.options.highlighting

`highlighting` - Configure which highlighter to use (and with which options) for each type of interaction.

List of highlighting interactions

- **default** - the default highlighter to use (and options) when none is specified
- **connecting** - when a valid link connection can be made to an element
- **embedding** - when a cell is dragged over another cell in embedding mode
- **magnetAvailability** - when showing all magnets to which a valid connection can be made
- **elementAvailability** - when showing all elements to which a valid connection can be made

Example usage:

```

new joint.dia.Paper({
  highlighting: {
    'default': {
      name: 'stroke',
      options: {
        padding: 3
      }
    },
    connecting: {
      name: 'addClass',
      options: {
        className: 'highlight-connecting'
      }
    }
  }
})

```

dia.Paper.prototype.options.interactive

`interactive` - if set to `false`, all interactions with elements and links are disabled. If it's defined as a function, the function is called with the element/link view (the user is about to interact with) as the first parameter followed by the name of the event (`'pointerdown'`, `'pointermove'`, ...). If the

returned value of such a function is `false` the interaction is disabled for this action. The returned value could be also defined as an object to achieve the fine-grained control over interaction. Here is the list of all properties of the `interactive` object to disable/enable only certain aspects of interactions. The properties not used within this object are enabled by default.

In the tables below there are lists of properties followed by an example with the related property set to `false`.

Links:

arrowheadMove		vertexMove	
vertexAdd		vertexRemove	
labelMove		useLinkTools	

Elements:

elementMove		addLinkFromMagnet	
--------------------	--	--------------------------	--

For a comparison the example below has all interactions on the link and on the elements enabled.

property `interactive` defaults to `{ labelMove: false }`

dia.Paper.prototype.options.linkConnectionPoint

`linkConnectionPoint(linkView, view, magnet, reference)` - this function allows you to customize what are the sticky points of links. The function must return a point (with `x` and `y` properties) where the link sticks to the element. The function takes the link view, element view, the magnet (SVG element) the link should stick to and a reference point (either the closest vertex or the sticky point on the other side of the link). Note that there is a utility function `shapePerimeterConnectionPoint` that can be directly passed to the `linkConnectionPoint` parameter. This function tries to find the best possible connection point right on the perimeter of the connected shapes. See the function documentation for details.

dia.Paper.prototype.options.linkPinning

`linkPinning` - when set to `true` (the default), links can be "pinned" to the paper meaning a source or target of a link can be a point. If you do not want to let the user to drag a link and drop it somewhere in a blank paper area, set this to `false`. The effect is that the link will be returned to its original position whenever the user drops it somewhere in a blank paper area.

dia.Paper.prototype.options.linkView

`linkView` - object that is responsible for rendering a link model into the paper. Defaults to `joint.dia.LinkView`. It can also be a function of the form `function(link)` that takes a link model and should return an object responsible for rendering that model onto the screen (in most cases a subtype of `joint.dia.LinkView`)

dia.Paper.prototype.options.markAvailable

`markAvailable` - marks all the available magnets or elements when a link is dragged (being reconnected). Default is `false`. This gives a hint to the user to what other elements/ports this link can be connected. What magnets/cells are available is determined by the `validateConnection` function. Internally, available magnets (SVG elements) are given the `'available-magnet'` class name and all the available cells the `'available-cell'` class name. This allows you to change the styling of the highlight effect.

dia.Paper.prototype.options.model

`model` - `joint.dia.Graph` object

dia.Paper.prototype.options.moveThreshold

`moveThreshold` - Number of required mousemove events before the first pointermove event is triggered. It defaults to `0`.

dia.Paper.prototype.options.multiLinks

`multiLinks` - when set to `false`, an element may not have more than one link with the same source and target element.

dia.Paper.prototype.options.origin

`origin` - position of zero coordinates of the paper in pixels (default is `{ x: 0, y: 0 }` i.e. top-left corner)

dia.Paper.prototype.options.perpendicularLinks

`perpendicularLinks` - if `true`, links will tend to get perpendicular to their associated objects. Default is `false`

dia.Paper.prototype.options.preventContextMenu

`preventContextMenu` - Prevents default context menu action (when right button of the mouse is clicked). It defaults to `true`.

dia.Paper.prototype.options.preventDefaultBlankAction

`preventDefaultBlankAction` - Prevents default action when an empty paper area is clicked. Setting the option to `false` will make the paper pannable inside a container on touch devices. It defaults to `true`.

dia.Paper.prototype.options.restrictTranslate

`restrictTranslate` - restrict the translation (movement) of elements by a given bounding box. If set to `true`, the user will not be able to move elements outside the boundary of the paper area. It's set to `false` by default. This option also accepts a function with signature `function(elementView)` in which case it must return a bounding box (an object of the form `{ x: Number, y: Number, width: Number, height: Number }`) that defines the area in which the element represented by `elementView` can be moved. For example, to restrict translation of embedded elements by the bounding box defined by their parent element, you can do:

```
restrictTranslate: function(elementView) {
  var parentId = elementView.model.get('parent');
  return parentId && this.model.getCell(parentId).getBBBox();
}
```

dia.Paper.prototype.options.snapLinks

`snapLinks` - when enabled, force a dragged link to snap to the closest element/port in the given radius. The option accepts `true` in which case default values are used or an object of the form `{ radius: <value> }` where you can specify the radius (default is 50).

dia.Paper.prototype.options.validateConnection

`validateConnection(cellViewS, magnetS, cellViewT, magnetT, end, linkView)` - decide whether to allow or disallow a connection between the source view/magnet (`cellViewS/magnetS`) and target view/magnet (`cellViewT/magnetT`). `end` is either `"source"` or `"target"` and tells which end of the link is being dragged. This is useful for defining whether, for example, a link starting in a port POut of element A can lead to a port Pin of element B. By default, all linkings are allowed.

dia.Paper.prototype.options.validateEmbedding

`validateEmbedding` - a function that allows you to control what elements can be embedded to what other elements when the paper is put into the `embeddingMode` (see above). The function signature is: `function(childView, parentView)` and should return `true` if the `childView` element can be embedded into the `parentView` element. By default, all elements can be embedded into all other elements (the function returns `true` no matter what).

dia.Paper.prototype.options.validateMagnet

`validateMagnet(cellView, magnet)` - decide whether to create a link if the user clicks a magnet. `magnet` is the DOM element representing the magnet. By default, this function returns `true` for magnets that are not explicitly set to "passive" (which is usually the case of input ports).

dia.Paper.prototype.options.width

`width` - width of the paper in pixels

env.addTest

`env.addTest`(name, fn)

Add a custom feature-detection test where `name` is a string which uniquely identifies your feature test and `fn` is a function that returns `true` if the browser supports the feature, and `false` if it does not.

```
joint.env.addTest('customTest', function() {
  // Just as an example, we will always return true here.
  return true;
});

if (joint.env.test('customTest')) {
  // Feature is supported.
}
```

env.test

`env.test`(name)

Tests whether the browsers supports the given feature or not. Returns `true` if the feature is supported, otherwise it returns `false`.

```
if (joint.env.test('someFeature')) {
  // Feature is supported.
}
```

JointJS ships with the following tests:

- **svgforeignobject** - Tests whether the browser supports [foreignObject](#).
-

highlighters

Highlighters can be used to provide visual emphasis to an element; during user interactions for example.

In the above demo, we listen to the paper for the `cell:pointerclick` event, then highlight the cell that was clicked.

```
paper.on('cell:pointerclick', function(cellView) {  
  cellView.highlight();  
});
```

As you can see it is possible to highlight a cell as follows:

```
cellView.highlight();
```

Unhighlighting a cell is simple too:

```
cellView.unhighlight();
```

To highlight a cell using a specific highlighter (and options):

```
cellView.highlight(null/* defaults to cellView.el */, {  
  highlighter: {  
    name: 'stroke',  
    options: {  
      width: 3  
    }  
  }  
});
```

Or you can specify just the highlighter's name (its default options will be used):

```
cellView.highlight(null/* defaults to cellView.el */, {  
  highlighter: 'stroke'  
});
```

highlighters.addClass

Toggles a class name on the cell view's `$el`.

Available options:

- **className** - the class name to toggle on the cell's `$el`

Example usage:

```
cellView.highlight(null/* defaults to cellView.el */, {  
  highlighter: {  
    name: 'addClass',  
    options: {  
      className: 'highlighted'  
    }  
  }  
});
```

```

    options: {
      className: 'some-custom-class'
    }
  }
});

```

highlighters.opacity

Changes the opacity of the cell view's `$el`.

When a cell is highlighted with the opacity highlighter, its `$el` is given the `.joint-highlight-opacity` class. To customize the look of this highlighted state, you can add custom CSS rules that affect this class name.

This highlighter does not currently have any options.

Example usage:

```

cellView.highlight(null/* defaults to cellView.el */, {
  highlighter: {
    name: 'opacity'
  }
});

```

highlighters.stroke

Adds a stroke around the cell view's `$el`.

Available options:

- **padding** - the space between the stroke and the element
- **rx** - the stroke's border radius on the x-axis
- **ry** - the stroke's border radius on the y-axis
- **attrs** - an object with SVG attributes to be set on the highlighter element

Example usage:

```

cellView.highlight(null/* defaults to cellView.el */, {
  highlighter: {
    name: 'stroke',
    options: {
      padding: 10,
      rx: 5,
      ry: 5,
      attrs: {
        'stroke-width': 3,
        stroke: '#FF0000'
      }
    }
  }
});

```

layout.DirectedGraph

Automatic layout of directed graphs. This plugin uses the open-source (MIT license) [Dagre](#) library internally. It provides a wrapper so that you can call the layout directly on JointJS graphs.

Note that you must include both the [Dagre](#) and [Graphlib](#) libraries as dependencies if you wish to use the `layout.DirectedGraph` plugin.

Usage

`joint.layout.DirectedGraph` plugin exposes the `joint.layout.DirectedGraph.layout(graphOrElements, opt)` function. The first parameter `graphOrElements` is the `joint.dia.Graph` or an array of `joint.dia.Elements` we want to layout. The second parameter `options` is an object that contains various options for configuring the layout.

```
var graphBBox = joint.layout.DirectedGraph.layout(graph, {
  nodeSep: 50,
  edgeSep: 80,
  rankDir: "TB"
});
console.log('x:', graphBBox.x, 'y:', graphBBox.y)
console.log('width:', graphBBox.width, 'height:', graphBBox.height);
```

A full blog post explaining this example in more detail can be found [here](#).

Example with clusters

Configuration

The following table lists options that you can pass to the `joint.layout.DirectedGraph.layout(graph, opt)` function:

nodeSep	a number of pixels representing the separation between adjacent nodes in the same rank
edgeSep	a number of pixels representing the separation between adjacent edges in the same rank
rankSep	a number of pixels representing the separation between ranks
rankDir	direction of the layout (one of <code>"TB"</code> (top-to-bottom) / <code>"BT"</code> (bottom-to-top) / <code>"LR"</code> (left-to-right) / <code>"RL"</code> (right-to-left))
marginX	number of pixels to use as a margin around the left and right of the graph.
marginY	number of pixels to use as a margin around the top and bottom of the graph.
ranker	Type of algorithm to assign a rank to each node in the input graph. Possible values: <code>'network-simplex'</code> (default), <code>'tight-tree'</code> or <code>'longest-path'</code> . number of pixels to use as a margin around the top and bottom of the graph.
resizeClusters	set to <code>false</code> if you don't want parent elements to stretch in order to fit all their embedded children. Default is <code>true</code> .
clusterPadding	A gap between the parent element and the boundary of its embedded children. It could be a number or an object e.g. <code>{ left: 10, right: 10, top: 30, bottom: 10 }</code> . It defaults to <code>10</code> .
setPosition(element, position)	a function that will be used to set the position of elements at the end of the layout. This is useful if you don't want to use the default <code>element.set('position', position)</code> but want to set the position in an animated fashion via transitions .
setVertices(link, vertices)	If set to <code>true</code> the layout will adjust the links by setting their vertices. It defaults to <code>false</code> . If the option is defined as a function it will be used to set the vertices of links at the end of the layout. This is useful if you don't want to use the default <code>link.set('vertices', vertices)</code> but want to set the vertices in an animated fashion via transitions .
setLabels(link, labelPosition, points)	If set to <code>true</code> the layout will adjust the labels by setting their position. It defaults to <code>false</code> . If the option is defined as a function it will be used to set the labels of links at the end of the layout. Note: Only the first label (<code>link.label(0)</code>) is positioned by the layout.

Additionally, the layout engine takes into account some properties on elements/links to fine tune the layout further. These are:

size	element	An object with <code>`width`</code> and <code>`height`</code> properties representing the size of the element.
minLen	link	The number of ranks to keep between the source and target of the link.
weight	link	The weight to assign edges. Higher weight edges are generally made shorter and straighter than lower weight edges.

labelPosition	link	Where to place the label relative to the edge. <code>'l'</code> = left, <code>'c'</code> = center (default), <code>'r'</code> = right.
labelOffset	link	How many pixels to move the label away from the edge. Applies only when <code>labelPosition</code> is left or right.
labelSize	link	The width and height of the edge label in pixels. e.g. <code>{ width: 100, height: 50 }</code>

The `layout()` function returns a bounding box (`g.Rect`) of the resulting graph.

API

The `layout.DirectedGraph` plugin extends the `joint.dia.Graph` type with functions that make it easy to convert graphs to and from the [Graphlib](#) graph format. This allows you to easily use many of the [graph algorithms](#) provided by this library.

toGraphLib()	Convert the JointJS <code>joint.dia.Graph</code> object to the Graphlib graph object. <pre>var graph = new joint.dia.Graph; // ... populated the graph with elements connected with links graphlib.alg.isAcyclic(graph.toGraphLib()) // true if acyclic</pre>
fromGraphLib(glGraph, opt)	Convert the Graphlib graph representation to JointJS <code>joint.dia.Graph</code> object and <code>opt.importEdge</code> are functions that accept a Graphlib node and edge return JointJS element/link. <pre>var g = new graphlib.Graph(); g.setNode(1); g.setNode(2); g.setNode(3); g.setEdge(1, 2); g.setEdge(2, 3); var graph = new joint.dia.Graph; graph.fromGraphLib(g, { importNode: function(node) { return new joint.shapes.basic.Rect({ position: { x: node.x, y: node.y }, size: { width: node.width, height: node.height } }); }, importEdge: function(edge) { return new joint.dia.Link({ source: { id: edge.v }, target: { id: edge.w } }); } });</pre>

layout.Port

Port layouts are functions that accept an array of port's `args` and return an array of port positions. Positions are relative to the element model bounding box. For example if we have an element at position `{ x:10, y:20 }` with a relative port position `{ x:1, y:2 }`, the absolute port position will be `{ x:11, y:22 }`.

Port layout can be defined only at the `group` level. Optionally you can pass some additional arguments into the layout function via `args`. The `args` is the only way how to adjust port layout from the port definition perspective.

```
var rect = new joint.shapes.basic.Rect({
  // ...
  ports: {
    groups: {
      'a': {
        position: {
          name: 'layoutType',
          args: {},
        }
      }
    },
    items: [
      // initialize 'rect' with port in group 'a'
      {
        group: 'a',
        args: {} // overrides `args` from the group level definition.
      },
      // ... other ports
    ]
  }
});

// ....
// add another port to group 'a'.
rect.addPort({ group: 'a' });
```

Pre-defined layouts:

On sides

A simple layout suitable for rectangular shapes. It evenly spreads ports along a single side.

name	string	Can be either <code>left</code> , <code>right</code> , <code>top</code> , <code>bottom</code> .		
args	object	x	number	Overrides the <code>x</code> value calculated by the layout function
		y	number	Overrides the <code>y</code> value calculated by the layout function
		dx	number	Added to the <code>x</code> value calculated by the layout function
		dy	number	Added to the <code>y</code> value calculated by the layout function
		angle	number	The port rotation angle.

```
{
  name: 'left',
  args: {
    x: 10,
    y: 10,
    angle: 30,
    dx: 1,
    dy: 1
  }
}
```

Line

A layout which evenly spreads ports along a line defined by a `start` and an `end` point.

name	string	<code>line</code>
------	--------	-------------------

args	object			
		start	{ x:number, y:number }	The line starting point
		end	{ x:number, y:number }	The line end point

```
{
  name: 'line',
  args: {
    start: { x: 10, y: 10 },
    end: { x: 20, y: 50 }
  }
}
```

Absolute

It lay a port out at the given position (defined as a `x`, `y` coordinates or percentage of the element dimensions).

name	string	absolute		
args	object	x	number string	Sets the port's <code>x</code> coordinate. Can be defined as a percentage string ('50%') or as a number
		y	number string	Sets the port's <code>y</code> coordinate. Can be defined as a percentage string ('50%') or as a number
		angle	number	The port rotation angle.

```
{
  name: 'absolute',
  args: {
    x: '10%',
    y: 10,
    angle: 45
  }
}
```

Radial

Suitable for circular shapes. The `ellipseSpreads` evenly spreads ports along an ellipse. The `ellipse` spreads ports from the point at `startAngle` leaving gaps between ports equal to `step`.

name	string	Can be either <code>ellipse</code> , <code>ellipseSpread</code> .		
args	object	x	number	Overrides the <code>x</code> value calculated by the layout function
		y	number	Overrides the <code>y</code> value calculated by the layout function
		dx	number	Added to the <code>x</code> value calculated by the layout function
		dy	number	Added to the <code>y</code> value calculated by the layout function
		dr	number	Added to the port delta rotation

		startAngle	number	Default value is <code>0</code> .
		step	number	Default <code>360 / portsCount</code> for the <code>ellipseSpread</code> , <code>20</code> for the <code>ellipse</code>
		startAngle	boolean	Set <code>compensateRotation:true</code> when you need to have ports in the same angle as an ellipse tangent at the port position.

```
{
  name: 'ellipseSpread',
  args: {
    dx: 1,
    dy: 1,
    dr: 1,
    startAngle: 10,
    step: 10,
    compensateRotation: false
  }
}
```

Custom layout

An alternative for built-in layouts is providing a function directly, where the function returns an array of port positions.

```
/**
 * @param {Array<object>} portsArgs
 * @param {g.Rect} elBBBox shape's bounding box
 * @param {object} opt Group options
 * @returns {Array<g.Point>}
 */
function(portsArgs, elBBBox, opt) {

  // ports on sinusoid
  return _.map(portsArgs, function(portArgs, index) {

    var step = -Math.PI / 8;
    var y = Math.sin(index * step) * 50;
    return g.point({ x: index * 12, y: y + elBBBox.height });

  });
}
```

Port layouts demo

layout.PortLabel

Port label layout functions calculate port label positions relatively to port positions.

Pre-defined port label layouts

On Sides

Simple label layout suitable for rectangular shapes. It places the label on arbitrary side of a port.

name	string	Can be either <code>left</code> , <code>right</code> , <code>bottom</code> , <code>top</code> .		
args	object string	x	number	Overrides the <code>x</code> value calculated by the layout function
		y	number	Overrides the <code>y</code> value calculated by the layout function
		angle	number	The port label rotation angle.
		attrs	number	JointJS style attribute applied on label's DOM element and it's children. The same notation as the <code>attrs</code> property on <code>Element</code> .

```
label: {
  position: {
    name : 'right',
    args: {
      x: 0,
      y: 0,
      angle: 0,
      attrs: {}
    }
  }
}
```

Inside/Outside

Places the label inside or outside of a rectangular shape. Where 'oriented' versions rotate the text towards the element center.

name	string	Can be either <code>inside</code> , <code>outside</code> , <code>insideOriented</code> , <code>outsideOriented</code> .		
args	object string	offset	number	Offset in direction from the shape's center.
		attrs	number	JointJS style attribute applied on label's DOM element and it's children. The same notation as the <code>attrs</code> property on <code>Element</code> .

```
label: {
  position: {
    name : 'outsideOriented',
    args: {
      offset: 10,
      attrs: {}
    }
  }
}
```

Radial

Places the label outside of a circular shape. Where the 'oriented' version rotates the text towards the element center.

name	string	Can be either <code>radial</code> , <code>radialOriented</code> .		
args	object string	offset	number	Offset in direction from the shape's center.
		attrs	number	JointJS style attribute applied on label's DOM element and it's children. The same notation as the <code>attrs</code> property on <code>Element</code> .

```
label: {
  position: {
    name : 'radialOriented',
    args: {
      offset: 0,
      attrs: {}
    }
  }
}
```

Manual placement

It allows to set a label position directly.

name	string	<code>manual</code>		
args	object string	x	number	Sets the label's <code>x</code> coordinate.
		y	number	Sets the label's <code>y</code> coordinate.
		angle	number	The port label rotation angle.
		attrs	number	JointJS style attribute applied on label's DOM element and it's children. The same notation as the <code>attrs</code> property on <code>Element</code> .

```
label: {
  position: {
    name: 'manual',
    args: {
      x: 10,
      y: 20,
      angle: 45,
      attrs: {}
    }
  }
}
```

Port label layout demo

shapes.devs

The Devs plugin provides you with ready-to-use shapes with predefined input & output port groups and simplified API.

joint.shapes.devs.Model

The Model shape implements simple API on top of the JointJS built-in ports. It splits ports into two semantic groups (**in** and **out**) and adds convenient methods for adding and removing ports.

inPorts	an array of all input ports
outPorts	an array of all output ports

shapes.devs.Model.addInPort

```
element.addInPort(port, [opt])
```

Add a single port into the `inPorts` array, where `port` is a name of the port.

shapes.devs.Model.addOutPort

```
element.addOutPort(port, [opt])
```

Add a single port into the `outPorts` array, where `port` is a name of the port.

shapes.devs.Model.changeInGroup

`element.changeInGroup(properties, [opt])`

Change the settings for the input ports, where `properties` is an object with a [group configuration](#). It extends the previous settings with the new configuration by default. Pass `{ rewrite: true }` via `opt` to invalidate the previous settings.

shapes.devs.Model.changeOutGroup

`element.changeOutGroup(properties, [opt])`

Change the settings for the output ports, where `properties` is an object with a [group configuration](#). It extends the previous settings with the new configuration by default. Pass `{ rewrite: true }` via `opt` to invalidate the previous settings.

shapes.devs.Model.removeInPort

`element.removeInPort(port, [opt])`

Remove a port from an element from the `inPorts` array, where `port` is a name of the port.

shapes.devs.Model.removeOutPort

`element.removeOutPort(port, [opt])`

Remove a port from an element from the `outPorts` array, where `port` is a name of the port.

joint.shapes.devs.Link

The `devs.Link` extends the `joint.dia.Link` and changes the link appearance.

Example usage

```
var shape = new joint.shapes.devs.Model({
  position: {
    x: 100,
    y: 100
  },
  inPorts: ['in1', 'in2'],
  outPorts: ['out1', 'out2']
});

shape.addTo(graph);

// adding/removing ports dynamically
shape.addInPort('in3');
shape.removeOutPort('out1').addOutPort('out3');

var link = new joint.shapes.devs.Link({
  source: {
    id: shape.id,
    port: 'out3'
  },
  target: {
    x: 200,
    y: 300
  }
});
link.addTo(graph);

// moving the input ports from `left` to `top`
shape.changeInGroup({ position: 'top' });
// moving the output ports `right` to `bottom`
shape.changeOutGroup({ position: 'bottom' });
```

Hierarchical diagrams

There are two more shapes designed for hierarchical diagrams as part of the plugin - `devs.Atomic` and `devs.Coupled`. They inherit from the `devs.Model` and differ only in the color and size. The purpose of these shapes is to enable you to implement a custom logic in your application based on their type. For instance a `devs.Coupled` can embed `devs.Atomic`'s but not the other way round as shown in the demo below.

When working with hierarchical diagrams there is a few methods, that might come in handy.

`coupled.embed(atomic)` that can put the ``atomic`` shape into the ``coupled``.

`coupled.fitEmbeds()` that resizes the ``coupled`` shape, so it visually contains all shapes embedded in.

`link.reparent()` that finds the best parent for the ``link`` based on the source and target element.

util.breakText

`util.breakText(text, size [, attrs, opt])`

Break the `text` into lines so that the text fits into the bounding box defined by `size.width` and, optionally, `size.height`. `attrs` is an object with SVG attributes that will be set on the SVG text before measuring it (such as `font-weight`, `font`, ...). If `opt.svgDocument` is specified, that SVG document will be used to append the internally created SVG text element used for measurements (by default, an SVG document is created automatically for you). This function creates a temporary SVG `<text>` element with the attributes provided and continuously measures its width and height trying to fit as many words as possible on each line. When the text overflows the `size.height`, it is cut. The return value is a string with automatically added newline characters (`\n`) so that the text fits into the bounding box specified. Useful for implementing auto-wrapping of text in JointJS shapes.

```
joint.util.breakText('this is quite a long text', { width: 50 })
// 'this is\nquite a\nlong\ntext'
```

util.cancelFrame

`util.cancelFrame(requestId)`

Cancels an animation frame request identified by `requestId` previously scheduled through a call to `joint.util.nextFrame`.

util.flattenObject

`util.flattenObject(object, delim, stop)`

Flatten a nested `object` up until the `stop` function returns `true`. The `stop` function takes the value of the node currently traversed. `delim` is a delimiter for the combined keys in the resulting object.

Example:

```
joint.util.flattenObject({
  a: {
    a1: 1,
    a2: 2,
    a3: {
      a31: 5,
      a32: {
        a321: { a3211: 5 }
      }
    }
  },
  b: 6
}, '/', function(v) { return !!v.a321; });

/*
{
  "a/a1": 1,
  "a/a2": 2,
  "a/a3/a31": 5,
  "a/a3/a32": {
    "a321": {
      "a3211": 5
    }
  },
  "b": 6
}
*/
```

util.getByPath

`util.getByPath(object, path, delim)`

Return a value at the `path` in a nested `object`. `delim` is the delimiter used in the `path`. Example:

```
joint.util.getByPath({ a: { aa: { aaa: 3 } } }, 'a/aa/aaa', '/');
// 3
```

util.getElementBBox

`util.getElementBBox(el)`

Return a bounding box of the element `el`. The advantage of this method is that it can handle both HTML and SVG elements. The resulting object is of the form `{ x: Number, y: Number, width: Number, height: Number }`.

util.guid

`util.guid()`

Return an identifier unique for the page.

util.imageToDataUri

```
util.imageToDataUri(url, callback)
```

Convert an image at `url` to the [Data URI scheme](#). This function is able to handle PNG, JPG and SVG formats. The `callback` function has the following signature `function(err, dataUri) {}`. Useful if you want to embed images right into your diagrams instead of having them referenced externally.

util.nextFrame

```
util.nextFrame(callback [, context])
```

Tell the browser to schedule the `callback` function to be called before the next repaint. This is a cross-browser version of the [window.requestAnimationFrame](#) function. Returns an ID of the frame request. For convenience, you can pass a `context` for your `callback` function.

util.normalizeSides

```
util.normalizeSides(box)
```

Return a new object of the form `{ top: Number, bottom: Number, left: Number, right: Number }`. The value for each of the side is determined by the argument `box`. If `box` is a number, all for sides will have this number. If it is an object with some/all of the `right`, `left`, `top`, `bottom` properties, these properties will be used. If any of the properties is missing, they will be set to `0` for the related side in the resulting object.

```
joint.util.normalizeSides(5) // { top: 5, left: 5, right: 5, bottom: 5 }
joint.util.normalizeSides({ left: 5 }) // { top: 0, left: 5, right: 0, bottom: 0 }
```

JointJS and Rappid use this method internally whenever there is an option object that can be specified either by a number or a (possibly incomplete) object with sides (for example, the `padding` option).

util.setAttributesBySelector

```
util.setAttributesBySelector(el, attrs)
```

Set attributes on the DOM element (SVG or HTML) `el` and its descendants based on the selector in the `attrs` object. The `attrs` object is of the form: `{ [selector]: {attributes} }`. For example:

```
var myEl = document.querySelector('.mydiv');
joint.util.setAttributesBySelector(myEl, {
  '.myDiv': { 'data-foo': 'bar' }, // Note the reference to the myEl element
  'input': { 'value': 'my value' }
});
```

The code above sets `'data-foo'` attribute of the `myEl` element to the value `'bar'` and `'value'` attribute of all the descendant inputs to the value `'my value'`. This is a convenient way of setting attributes on elements (including themselves) and their descendants. Note that `'class'` attribute is treated as a special case and its value does not override the previous value of the `'class'` attribute. Instead, it adds this new class to the list of classes for the element referenced in the selector.

util.setByPath

`util.setByPath(object, path, value, delim)`

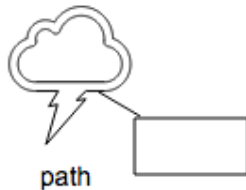
Set a `value` at the `path` in a nested `object`. `delim` is the delimiter used in the `path`. Returns the augmented `object`.

```
joint.util.setByPath({ a: 1 }, 'b/bb/bbb', 2, '/');
/*
{
  "a": 1,
  "b": {
    "bb": {
      "bbb": 2
    }
  }
}
*/
```

util.shapePerimeterConnectionPoint

`util.shapePerimeterConnectionPoint(linkView, view, magnet, ref)`

This function can be directly used in the `dia.Paper` `linkConnectionPoint` parameter. When used, links will try to find the best connection point right on the perimeter of the connected shape rather than only on the bounding box. See the image below.



```
var paper = new joint.dia.Paper({
  // ...
  linkConnectionPoint: joint.util.shapePerimeterConnectionPoint
  // ...
})
```

util.sortElements

`util.sortElements(elements, comparator)`

Change the order of `elements` (a collection of HTML elements or a selector or jQuery object) in the DOM according to the `comparator(elementA, elementB)` function. The `comparator` function has the exact same meaning as in `Array.prototype.sort(comparator)`.

util.toggleFullScreen

`util.toggleFullScreen([element])`

Make (or cancel) an `element` to be displayed full-screen. The default element is `window.top.document.body`.

```
joint.util.toggleFullScreen(paper.el);
```

util.unsetByPath

`util.unsetByPath(object, path, delim)`

Unset (delete) a property at the `path` in a nested `object`. `delim` is the delimiter used in the `path`. Returns the augmented `object`.

```
joint.util.unsetByPath({ a: { aa: { aaa: 3 } } }, 'a/aa/aaa', '/');  
// { a: { aa: { } } }
```

util.uuid

`util.uuid()`

Return a pseudo-UUID.

util.format.number

`util.format.number(specifier, value)`

Format number `value` according to the `specifier` defined via the [Python Format Specification Mini-language](#).

```
joint.util.format.number('.2f', 5)      // 5.00  
joint.util.format.number('03d', 5)     // 005  
joint.util.format.number('.1%', .205)   // 20.5%  
joint.util.format.number('*^9', 5)      // ****5****
```